

15.7.19

Unit-I
 Reference Book:
 P. Raja, Omesh
 Publications
Functions:

- malloc() → used to allocate memory at run time
- free() → used to deallocate memory at run time

These two func^{ns} are supported by header files:
 stdlib.h, malloc.h, alloc.h (any one can be used)

Q1. WAP to implement the dynamic memory allocation.

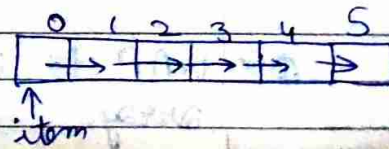
Ans1. #include <malloc.h>
 void main()
 { int *p, m, i, sum=0;
 printf("Enter the no. of elements");
 scanf("%d", &m);
 p = (int*) malloc (m * size of (m));
 ↳ Syntax of dynamic memory allocation
 printf("Enter the elements");
 for (i=0; i<m; i++)
 { scanf("%d", (p+i));
 printf("%d", *(p+i));
 }
 for (i=0; i<m; i++)
 sum = sum + *(p+i);
 printf("%d", sum);
 free(p);
 getch();
 }

Q2. WAP to implement a structure with pointer.

Ans2. #include <stdio.h>
 struct num
 { int m₁, m₂; }
 void main()
 { struct num m;
 struct num *p;
 p = &m;
 printf("Enter values");
 scanf("%d %d", &p->m₁, &p->m₂);
 printf("%d %d", p->m₁, p->m₂);
 }

Q3. WAP to insert an element in the beginning & end.

Ans3. void main()
 {
 int a[20], len, val, i, item;
 printf("Enter length of array:");
 scanf("%d", &len);
 for (i=0; i<len; i++)
 { printf("Enter the %d element", i+1);
 scanf("%d", &val);
 a[i] = val;
 }
 printf("Enter the element you want to insert in the beginning");
 scanf("%d", &item);
 for (i=len; i>=0; i--)
 { a[i] = a[i-1];
 }




```

a[0] = item;
m++;
for (i=0; i < m; i++)
    printf("%d", a[i]);
printf("Enter value at end:");
scanf("%d", &val);
a[m] = val;
m++;
}

```

Q4. WAP to insert an element at a given location.

```

Ans 4. void main()
{
    int a[100], m, i, loc, val;
    printf("Enter no. of elements");
    scanf("%d", &m);
    for (i=0; i < m; i++)
    {
        scanf("%d", &a[i]);
    }
    printf("Enter the location & element:");
    scanf("%d %d", &loc, &val);
    for (i=m; i >= loc; i--)
    {
        a[i] = a[i-1];
    }
    a[loc] = val;
    m++;
    for (i=0; i < m; i++)
        printf("%d", a[i]);
}

```

Q5. WAP to insert an element in a sorted array.

```

Ans 4. void main()
{
    int m, i, val, a[100], pos;
    printf("Enter no. of elements");
    scanf("%d", &m);
    for (i=0; i < m; i++)
        scanf("%d", &a[i]);
    printf("Enter values you want to insert");
    scanf("%d", &val);
    for (i=0; i < m; i++)
    {
        if (a[i] > val)
            pos = i;
    }
    for (i=m; i >= pos; i--)
    {
        a[i] = a[i-1];
    }
    a[pos] = val;
    m++;
    for (i=0; i < m; i++)
        printf("%d", a[i]);
}

```

Q5. WAP to delete first & last element of an array.

```

Ans 5. void main()
{
    int a[100], m, i;
    printf("Enter no. of elements");
    scanf("%d", &m);
    for (i=0; i < m; i++)
        scanf("%d", &a[i]);
    for (i=0; i < m; i++)
    {
        a[i] = a[i+1];
    }
    a[0] = NULL;
}

```



```

    m--;
}
a[m-1] = NULL;
m--;

```

Q6. WAP to delete a particular element from an array.

Ans 6. void main()

```

{ int a[100], m, i, data, loc;
  printf("Enter no. of elements");
  scanf("%d", &m);
  for (i=0; i<m; i++)
    scanf("%d", &a[i]);
  printf("Enter element you want to delete");
  scanf("%d", &data);
  for (i=0; a[i] != data; i++)
    loc = i;
  for (i=loc; i<m; i++)
  { a[i] = a[i+1];
    a[i] = NULL;
    m--;
  }
}

```

25/7/19

Array Insertion & Deletion:

```

void iab(), iae()
void main()
{ int a[100], i, m, ch;
  printf("Enter no. of elements");
  scanf("%d", &m);
  printf("Enter Elements");

```

```

for (i=0; i<m; i++)
  scanf("%d", &a[i]);
  printf("1.) Insertion at begin");
  printf("Enter your choice

```

```

printf("Enter your choice");
scanf("%d", &ch);
if (ch == 1)
{ iab();
  disp();
}
else
if (ch == 2)
{ iae();
  disp();
}
}

```

```

}
void iab()
{ int item;
  printf("Enter element");
  scanf("%d", &item);
  for (i=0; i<m; i++)
    a[i] = a[i+1];
  a[i] = item;
  m++;
}

```


Data Structure :

- Data Structure is referred to the organisation of data in computer memory or the way in which data is efficiently stored, processed & repeat is called the data structure.
- Data structure is the structured representation of logical relationships b/w elements of data.
- There are different types of data structures some are used to store the data of same type & some are used to store the data of different type.
- Data structure is not only ~~only~~ deals with raw data but also involve the relationship b/w data elements.

Types of Data :

I According to the nature of size :

1. Static Data Structure : A data structure is said to be static if we can store data upto a fixed number. Eg: Array.
2. Dynamic Data Structure : A dynamic data structure is that which allows the programmer to change its size during the execution of program to add or delete the data.

II According to its occurrence :

1. Linear data structure : In linear data structure the data is stored in consecutive memory location or in sequential manner. Eg: Array, linked list, stack, queue.

2. Non-linear data structure : In Non-linear data structure the data is stored in non-consecutive ~~manner~~ memory location or in sequential form. Eg: tree & graph.

III Homogeneous & Non Homogeneous :

1. Homogeneous data structure : In homogeneous data structure the elements are of same type. Eg: array.

2. Non-Homogeneous data structure : In non-homogeneous data structure the elements are of different type. Eg: structure, union.

IV Primitive & Non-Primitive :

1. Primitive : Primitive data structure are basic data structure that are directly ~~off~~ operated upon by the machine instruction. These have different representation on different computers. Eg: Integer, float, character, pointer.

2. Non-Primitive : Non-primitive data structures are more complicated emphasizing on structuring of a group of homogeneous & non-homogeneous data forms. Eg: Array, linked-

list, tree, graph.

Data Structure Operation:

1. Creating: This is the first operation of data structure. This is just the declaration & initialization of data structure.
2. Inserting: Adding the new record to a data structure.
3. Deleting: Removing a record from the structure.
4. Updating: It changes data values of data structure.
5. Traversing: Accessing of each record exactly once.
6. Searching: Finding the location of record with a given key value.
7. Sorting: Arranging the data elements in some logical order i.e. ascending or descending.
8. Merging: Combine the data elements in two different ~~sorted~~ sorted set into a single sorted set.
9. Destroying: This is the last operation of data structure & apply this operation when no longer

needs of data.

Algorithms:

1.) Traversing:

Step-1: $I \rightarrow$ Lower bound

Step-2: for $I = LB$ to UB (Upper bound)
print $A[I]$

Step-3: Stop

2.) Insertion at begin:

Step-1: If $UB = \text{Max}$. then write overflow & stop

Step-2: Read data

Step-3: $K \rightarrow UB$

Step-4: Repeat Step-5 while $K \geq LB$

Step-5: $A(K+1) \rightarrow A(K)$
 $K \rightarrow K-1$

Step-6: $A(LB) = \text{Data}$

Step-7: Stop

3.) Insertion at End:

Step-1: If $UB = \text{Max}$ then write overflow & stop.

Step-2: Read data

Step-3: $UB = UB + 1$
 $A[UB] = \text{Data}$

Step-4: Stop

✓ 4.) Insertion at given location:

Step-1: If $UB = \text{Max}$ then write overflow & stop

Step-2: Read data

Step-3: Read location

Step-4: $K = UB$

Step-5: Repeat step-6 while $K \geq \text{location}$

Step-6: $A(K+1) = A(K)$
 $K = K - 1$

Step-7: $A(\text{location}) = \text{Data}$

Step-8: Stop

5.) Insertion in Sorted Array:

Step-1: If $UB = \text{Max}$ then write overflow & stop

Step-2: Read data

Step-3: $K = LB$

Step-4: Repeat Step-5 while $A(K) < \text{Data}$

Step-5: $K = K + 1$

Step-6: $\text{Location} = K$
 $K = UB$

Step-7: Repeat Step-8 while $K \geq \text{location}$

Step-8: $A(K+1) = A(K)$
 $K = K - 1$

Step-9: $A(\text{location}) = \text{Data}$

Step-10: Stop

6.) Deletion from beginning:

Step-1: If $N = 0$, then write underflow & stop.

Step-2: $K = L.B$

Step-3: Repeat step-4 while $K \leq U.B$

Step-4: $A[K] = A[K+1]$

Step-5: $A[UB] = \text{NULL}$
 $UB = UB - 1$

Step-6: Stop

7.) Deletion from a particular location:

Step-1: If $N=0$, then write underflow & stop.

Step-2: Read data.

Step-3: $K = LB$

Step-4: Repeat Step-5 while $A[K] \neq \text{data}$

Step-5: $K = K+1$

Step-6: $LOC = K$

Step-7: Repeat Step-8 while $K < UB$

Step-8: $A[K] = A[K+1]$, $K = K+1$

Step-9: $A[UB] = \text{NULL}$
 $UB = UB-1$

Step-10: Stop

8.) Concatenation:

Step-1: $K = LBA$ (Lower Bound of A)

Step-2: Repeat Step-3 while $K \leq UBA$

Step-3: $C[K] = A[K]$
 $K = K+1$

Step-4: $L = LBB$ (Lower Bound of B)

Step-5: Repeat Step-6 while $L \leq UBB$

Step-6: $C[K] = B[L]$
 $L = L+1$, $K = K+1$

Step-7: Stop

9.) Merging:

Step-1: $I = LBA$, $J = LBB$, $K = 0$

Step-2: Repeat Step-3 & 4 while $I \leq UBA$
& $J \leq UBB$

Step-3: If $A(I) < B(J)$, then
 $C[K] = A[I]$
 $I = I+1$
else
 $C[K] = B[J]$
 $J = J+1$

Step-4: $K = K+1$

Step-5: Repeat Step-6 while $I \leq UBA$

Step-6: $C[K] = A[I]$
 $I = I+1$, $K = K+1$

Step-7: Repeat Step-8 while $J \leq UBB$

Step-8: $C[K] = B[J]$

$$J = J + 1, K = K + 1$$

Step-9: Stop

To Find the address of a particular element in 1D & 2D array:

- 1D:
1. Length = $UB - LB + 1$
 2. Loc $A[K] = \text{Base}(A) + W * K$
Loc $A[K] = \text{Base}(A) + W * (K - LB)$

Q1. Find the address of 4th element in an array $A[10]$ with Base address = 400 & each element occupies 2 bytes.

Ans1. $\text{Base}(A) = 400$
 $W = 2$
 $\Rightarrow A[4] = 400 + 2 * 4$
 $= 408$

Q2. Find the number of elements in linear array (i) $A(5:50)$, (ii) $B(-5:10)$, (iii) $C[18]$

Ans2 (i) Length $(A) = 50 - 5 + 1$
 $= 46$

(ii) Length $(B) = 10 - (-5) + 1$
 $= 16$

(iii) Length $(C) = 18 - 0 + 1$
 $= 19$

Q2(b) Find the address of elements $A[15]$, $A[40]$ & $A[55]$ with base address = 300 & each element occupies 4 bytes & lower bound = 5.

Ans2(b) (i) $A[15] = 300 + 4 * (15 - 5)$
 $= 340$

(ii) $A[40] = 300 + 4 * (40 - 5)$
 $= 440$

(iii) $A[55] = 300 + 4 * (55 - 5)$
 $= 500$

→ 2D array:

Row Major Order (RMO):

1	2	3
4	5	6
7	8	9

Column Major Order (CMO):

1	4	7
2	5	8
3	6	9

RMO Formula:

Location $A[i][j] = \text{Base}(A) + W [m(i - LB \text{ for row index}) + (j - 1 - LB \text{ for column index})]$

where $m \rightarrow$ no. of columns.

$W \rightarrow$ no. of bytes required by each element

CMO Formula:

Location $A[i][j] = \text{Base}(A) + W[(i - L.B \text{ for row index}) + m(j - L.B \text{ for column index})]$

where $m \rightarrow$ no. of rows

Q1. Find the location of element $A[10][10]$ in an array $A[20][50]$ with base address = 2000, each element requires 4 bytes & lower bound = 1.

Ans1. RMO:

$$\begin{aligned} \text{Location } A[10][10] &= 2000 + 4[50(10-1) + (10-1)] \\ &= 2000 + 4[450 + 9] \\ &= 2000 + 1836 \\ &= 3836 \end{aligned}$$

CMO:

$$\begin{aligned} \text{Location } A[10][10] &= 2000 + 4[(10-1) + 20(10-1)] \\ &= 2000 + 4[9 + 180] \\ &= 2000 + 756 \\ &= 2756 \end{aligned}$$

Q2. Find the address of element $X[4,3]$ in an array $X[1 \dots 5, 1 \dots 4]$ stored in row major order (RMO) with base address = 1000 & each element requires 4 bytes.

Ans2. $M = 5 - 1 + 1 = 5$
 $N = 4 - 1 + 1 = 4$

RMO:

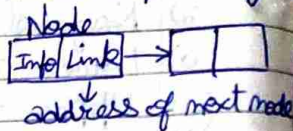
$$\begin{aligned} \text{Location of } X[4][3] &= 1000 + 4[4(4-1) + (3-1)] \\ &= 1000 + 4[12 + 2] \\ &= 1000 + 56 \\ &= 1056 \end{aligned}$$

Q1. WAP to print the following matrix.

- Diagonal matrix
- Lower triangle
- Upper triangle
- Tridiagonal
- Multiplication of two matrices

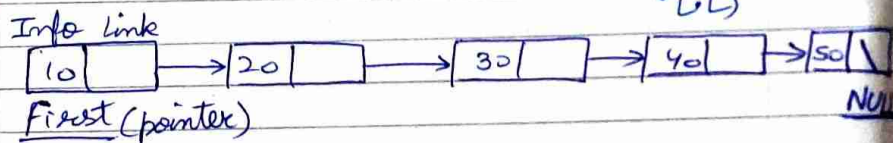
29.7.19 Linked Lists:

→ Linked List is the ordered collection of finite homogeneous data elements called the nodes where the linear order is maintained by the pointers i.e. each node is divided into two parts, first part contains the information of elements & the second part is called the Link field which contains the address of next node.
→ The number of pointers is maintained depending upon the requirement & usage.



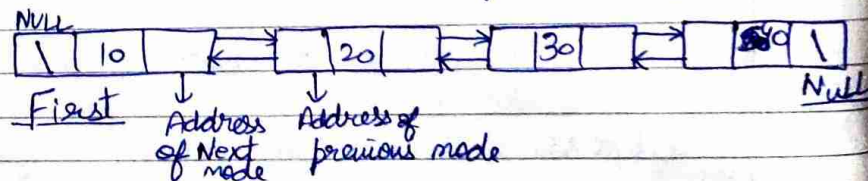
Types of Linked Lists:

1.) Singly Linked List: Info | Link (also known as ground headed LL)



Singly Linked List can have multiple nodes

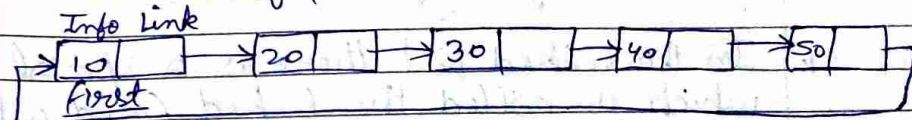
2.) Doubly Linked List: LPT | Info | RPT
Left pointer Right pointer



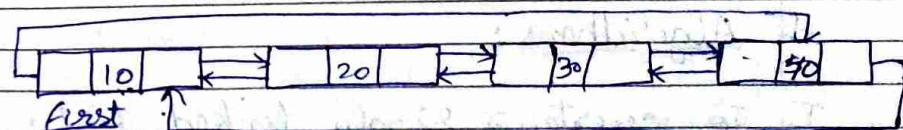
3.) Circular Singly Linked List: (Circular headed linked list)

In Circular Singly linked list last node contains

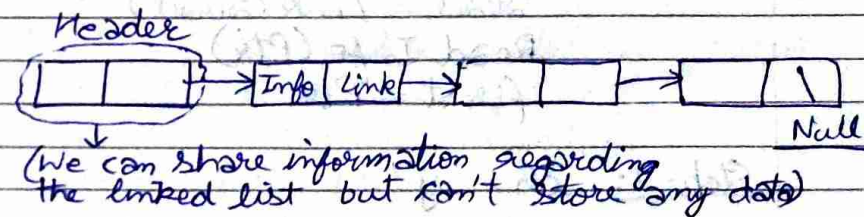
the address of first one.



4.) Circular Doubly Linked List:



5.) Header Linked List:



Advantages of Linked List:

1. Linked list are the dynamic data structure they can extend or reduce at one time.
2. The size of the linked list is not fixed.
3. Data can store non-continuous memory box.
4. Insertion & deletion of nodes are easier & efficient.
5. Many more complex applications can easily implemented with linked list.

Disadvantages of Linked List:

1. In the linked list there is a special field which is called the linked field which holds the address of next node so, linked list requires extra space

Algorithms:

I To create a singly linked list:

Step-1: Ptc = avail
avail = Link (avail)
Read Info (Ptc)
first = Ptc

Step-2: ch = 'y'

Step-3: Repeat Step-4 & Step-5
while ch = 'y'

Step-4: Cpt = avail
avail = Link (avail)
Read Info (Cpt)
Link (Ptc) = Cpt
Ptc = Cpt

Step-5: Read choice Y/N into ch for more nodes

Step-6: Link (Ptc) = NULL

Step-7: Stop

Q1. WAP to create a singly linked list.

Ans: struct node
{ int info;
struct node * Link;
}; char ch;
struct node * first, * ptc, * cpt;
void main()
{ ptc = (struct node *) malloc (size of (struct node));
printf ("Enter first node information");
scanf ("%d", & ptc → info);
first = ptc;
do
{ cpt = (struct node *) malloc (size of (struct node));
printf ("Enter next node info");
scanf ("%d", & cpt → info);
ptc → Link = cpt;
ptc = cpt;
printf ("Do you want to add more nodes");
scanf ("%c", & ch);
} while (ch == 'y' || ch == 'Y');
ptc → Link = NULL;

II Traversal:

Step-1: Ptc = first

Step-2: Repeat Step-3 to Step-4
while Ptc != NULL

Step-3: Print info (Ptc)

Step-4: $P_{tr} = \text{Link}(P_{tr})$

Step-5: Stop

```
void display()
{
    ptr = first;
    while (ptr != NULL)
    {
        printf("%d", ptr -> info);
        ptr = ptr -> link;
    }
}
```

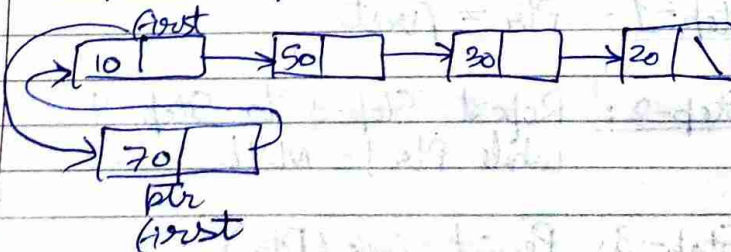
1.8.19 III Insertion at begin:

Step-1: If $AVAIL = NULL$
then write overflow & stop.

Step-2: $P_{tr} = AVAIL$
 $AVAIL = \text{Link}(AVAIL)$
Read Info(P_{tr})

Step-3: $\text{Link}(P_{tr}) = \text{first}$
 $\text{first} = P_{tr}$

Step-4: Stop



Program:

```

ptr = (struct node *) malloc (size of (struct node));
printf("Enter new node info");
scanf("%d", &ptr -> info);
ptr -> link = first;
first = ptr;

```

IV Insertion at Last:

Step-1: If $AVAIL = NULL$
then write overflow & stop

Step-2: $P_{tr} = AVAIL$
 $AVAIL = \text{Link}(AVAIL)$
Read info(P_{tr})

Step-3: $C_{pt} = \text{first}$

Step-4: Repeat step-5 while $\text{Link}(C_{pt}) \neq NULL$

Step-5: $C_{pt} = \text{Link}(C_{pt})$

Step-6: $\text{Link}(C_{pt}) = P_{tr}$
 $\text{Link}(P_{tr}) = NULL$

Step-7: Stop

V Insertion after given info of node:

Step-1: If $AVAIL = NULL$
then write overflow & stop

Step-2: $Ptc = Avail$
 $Avail = Link(Avail)$
Read info(Ptc)

Step-3: Read data after which you want to insert the node.

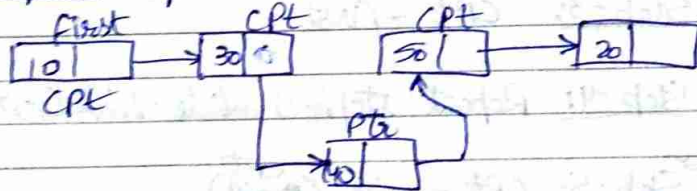
Step-4: $Cpt = first$

Step-5: Repeat Step-6 while $Info(Cpt) \neq Data$

Step-6: $Cpt = Link(Cpt)$

Step-7: $Link(Ptc) = Link(Cpt)$
 $Link(Cpt) = Ptc$

Step-8: Stop



VI Insertion in Sorted List:

Step-1: If $Avail = NULL$
then write overflow & stop

Step-2: $Ptc = Avail$
 $Avail = link(Avail)$
Read Info(Ptc)

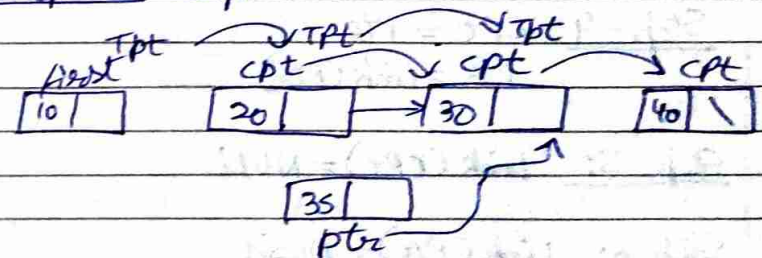
Step-3: $Cpt = first$

Step-4: Repeat Step-5 while
 $Info(Cpt) < Info(Ptc)$

Step-5: $Tpt = Cpt$
 $Cpt = Link(Cpt)$

Step-6: $Link(Tpt) = Ptc$
 $Link(Ptc) = Cpt$

Step-7: Stop



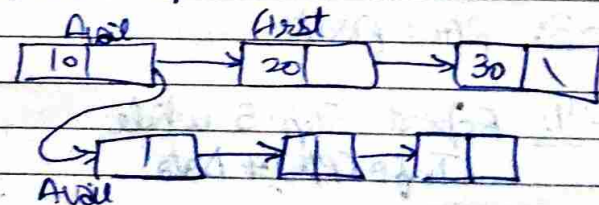
VII Deletion from beginning:

Step-1: If $first = NULL$
then write underflow & stop

Step-2: $Ptc = first$
 $first = Link(Ptc)$

Step-3: $Link(Ptc) = Avail$
 $Avail = Ptc$

Step-4: Stop



VIII Deletion from End:

Step-1: If $\text{first} = \text{NULL}$
then write underflow & stop

Step-2: $\text{Ptr} = \text{first}$

Step-3: Repeat Step-4 while
 $\text{Link}(\text{Ptr}) \neq \text{NULL}$

Step-4: $\text{Cpt} = \text{Ptr}$
 $\text{Ptr} = \text{Link}(\text{Ptr})$

Step-5: $\text{Link}(\text{Cpt}) = \text{NULL}$

Step-6: $\text{Link}(\text{Ptr}) = \text{Avail}$
 $\text{Avail} = \text{Ptr}$

Step-7: Stop.

IX Deletion of Particular Node:

Step-1: If $\text{first} = \text{NULL}$
then write underflow & stop

Step-2: Read Data which node you want
to remove.

Step-3: $\text{Ptr} = \text{first}$

Step-4: Repeat Step-5 while
 $\text{Info}(\text{Ptr}) \neq \text{Data}$

Step-5: $\text{Cpt} = \text{Ptr}$
 $\text{Ptr} = \text{Link}(\text{Ptr})$

Step-6: $\text{Link}(\text{Cpt}) = \text{Link}(\text{Ptr})$

Step-7: $\text{Link}(\text{Ptr}) = \text{Avail}$
 $\text{Avail} = \text{Ptr}$

Step-8: Stop

Program (Deletion):

```
void DPEC()
{
    printf("Enter data which you want to remove");
    scanf("%d", &data);
    Ptr = first;
    while (Ptr->info != data)
    {
        Cpt = Ptr;
        Ptr = Ptr->Link;
    }
    Cpt->Link = Ptr->Link;
    free(Ptr);
}
```

Circular Singly Linked List:



I Traversing:

Step-1: $P_{tr} = \text{first}$
Process info (P_{tr})
 $P_{tr} = \text{Link}(P_{tr})$

Step-2: Repeat Step-3 while $\text{Link}(P_{tr}) \neq \text{first}$

Step-3: Process info (P_{tr})
 $P_{tr} = \text{Link}(P_{tr})$

Step-4: Stop

2.8.19 II Insertion at beginning:

Step-1: If $\text{avail} = \text{NULL}$ then write overflow & stop

Step-2: $P_{tr} = \text{avail}$
 $\text{avail} = \text{Link}(\text{avail})$
Read info (P_{tr})

Step-3: $C_{pt} = \text{first}$

Step-4: Repeat Step-5 while $\text{Link}(C_{pt}) \neq \text{first}$

Step-5: $C_{pt} = \text{Link}(C_{pt})$

Step-6: $\text{Link}(P_{tr}) = \text{first}$
 $\text{first} = P_{tr}$
 $\text{Link}(C_{pt}) = \text{first}$

Step-7: Stop

III Insertion at End:

Step-1: If $\text{avail} = \text{NULL}$ then write overflow & stop

Step-2: $P_{tr} = \text{avail}$
 $\text{avail} = \text{Link}(\text{avail})$
Read Info (P_{tr})

Step-3: $C_{pt} = \text{first}$

Step-4: Repeat Step-5 while $\text{Link}(C_{pt}) \neq \text{first}$

Step-5: $C_{pt} = \text{Link}(C_{pt})$

Step-6: $\text{Link}(C_{pt}) = P_{tr}$
 $\text{Link}(P_{tr}) = \text{first}$

Step-7: Stop

IV Deletion from beginning:

Step-1: If $\text{first} = \text{NULL}$ then write underflow & stop

Step-2: $C_{pt} = \text{first}$

Step-3: Repeat Step-4 while $\text{Link}(C_{pt}) \neq \text{first}$

Step-4: ~~$C_{pt} = \text{first}$~~ $C_{pt} = \text{Link}(C_{pt})$

Step-5: $Ptr = first$
 $first = Link(Ptr)$
 $Link(Cpt) = first$

Step-6: $Link(Ptr) = avail$
 $avail = Ptr$

Step-7: Stop

V Deletion from end:

Step-1: If $first = NULL$ then write underflow & stop

Step-2: $Cpt = first$

Step-3: Repeat Step-4 while
 $Link(Cpt) \neq first$

Step-4: $Ptr = Cpt$
 $Cpt = Link(Cpt)$

Step-5: $Link(Ptr) = first$

Step-6: $Link(Cpt) = avail$
 $avail = Cpt$

Step-7: Stop

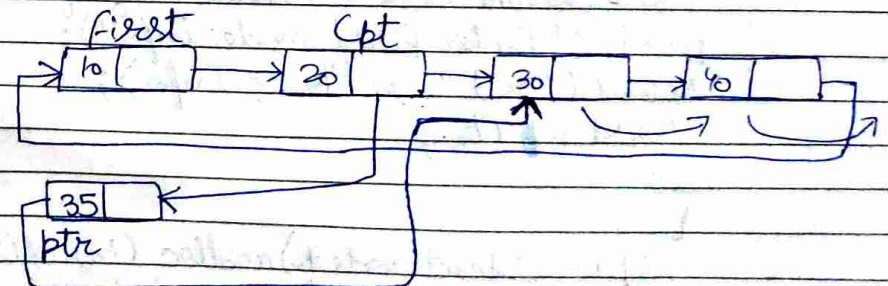
Program: (Deletion)

```
void DFE()
{ Cpt = first;
```

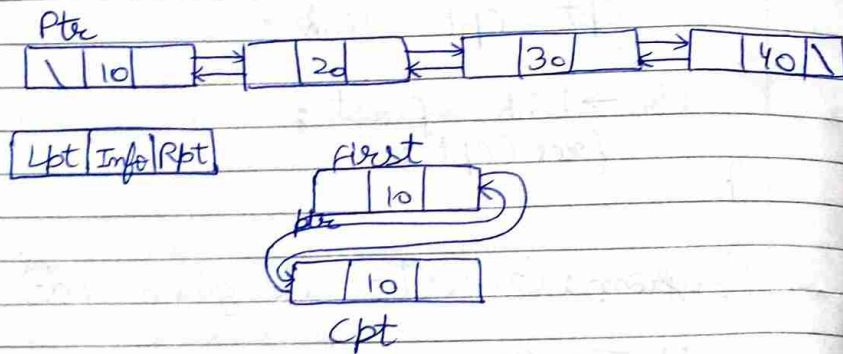
```
while (Cpt -> Link != first)
{ Ptr = Cpt;
  Cpt = Cpt -> Link;
}
Ptr -> Link = first;
free(Cpt);
}
```

Program: (Insertion after given node)

```
void I(GNC)
{ Ptr = (struct mode *) malloc (Size of (struct mode));
  printf("Enter info of mode");
  scanf("%d", &Ptr -> Info);
  printf("Enter data after which you want to insert");
  scanf("%d", &Data);
  Cpt = first;
  while (Cpt -> Info != Data)
  { Cpt = Cpt -> Link;
    Ptr -> Link = Cpt -> Link;
    Cpt -> Link = Ptr;
  }
}
```



Doubly Linked List:



Program: (Creation of Doubly Linked list)

```
struct node
{
```

```
    int info;
    struct node *Lpt, *Rpt;
};
```

```
struct node *first, *Pte, *Cpt;
```

```
void main()
{
```

```
    char ch;
```

```
    Pte = (struct node *) malloc (size of (struct node))
```

```
    printf("Enter first node info");
```

```
    scanf("%d", &Pte->Info);
```

```
    first = Pte;
```

```
    do
```

```
    {
```

```
        Cpt = (struct node *) malloc (size of (struct node))
```

```
        printf("Enter next node info");
```

```
        scanf("%d", &Cpt->Info);
```

```
        Pte->Rpt = Cpt;
```

```
        Cpt->Lpt = Pte;
```

```
        Pte = Cpt;
```

```
        printf("Do you want to add more Nodes  
Y/N");
```

```
        scanf("%c", &ch);
```

```
    } while (ch == 'Y' || ch == 'y');
```

```
    Pte->Rpt = NULL;
```

```
    first->Lpt = NULL;
```


5.8-19

(I) Traversal : (Forward)

Step-1: $P_{tr} = \text{first}$

Step-2: Repeat Step-3 while
 $P_{tr} \neq \text{NULL}$

Step-3: Process info (P_{tr})
 $P_{tr} = \text{RPT}(P_{tr})$

Step-4: Stop

II Traversal : (Backward)

Step-1: $P_{tr} = \text{first}$

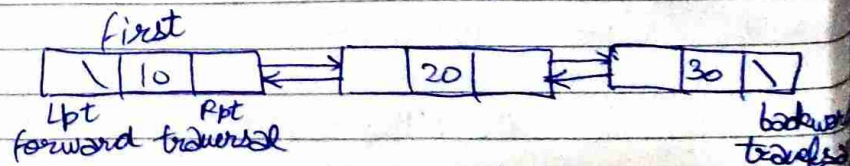
Step-2: Repeat Step-3 while
 $\text{Rpt}(P_{tr}) \neq \text{NULL}$

Step-3: $P_{tr} = \text{Rpt}(P_{tr})$

Step-4: Repeat Step-5 while
 $P_{tr} \neq \text{NULL}$

Step-5: Process Info (P_{tr})
 $P_{tr} = \text{Lpt}(P_{tr})$

Step-6: Stop.



III Insertion at Begin:

Step-1: If $\text{avail} = \text{NULL}$ then write overflow & stop

Step-2: $P_{tr} = \text{avail}$
 $\text{avail} = \text{Rpt}(\text{Cavail})$
Read info (P_{tr})

Step-3: $\text{Rpt}(P_{tr}) = \text{first}$
 $\text{Lpt}(\text{first}) = P_{tr}$
 $\text{first} = P_{tr}$
 $\text{Lpt}(P_{tr}) = \text{NULL}$

Step-4: Stop

IV Insertion at End:

Step-1: If $\text{avail} = \text{NULL}$ then write overflow & stop

Step-2: $P_{tr} = \text{avail}$
 $\text{avail} = \text{Rpt}(\text{Cavail})$
Read info (P_{tr})

Step-3: $\text{Cpt} = \text{first}$

Step-4: Repeat Step-5 while
 $\text{Rpt}(\text{Cpt}) \neq \text{NULL}$

Step-5: $\text{Cpt} = \text{Rpt}(\text{Cpt})$

Step-6: $\text{Rpt}(\text{Cpt}) = P_{tr}$
 $\text{Lpt}(P_{tr}) = \text{Cpt}$

~~Insertion in linked list:~~ $Rpt(Ptr) = NULL$

Step-7: Stop

II Insertion After given info of node:

Step-1: If avail = NULL then write overflow & stop.

Step-2: $Ptr = avail$
 $avail = Rpt(avail)$
Read info (Ptr)

Step-3: Read Data

Step-4: $Cpt = first$

Step-5: Repeat Step-6 while
 $Info(Cpt) \neq Data$

Step-6: $Cpt = Rpt(Cpt)$

Step-7: $Tpt = Rpt(Cpt)$

Step-8: $Rpt(Tpt) = Tpt$
 $Lpt(Tpt) = Ptr$
 $Rpt(Cpt) = Ptr$
 $Lpt(Ptr) = Cpt$

Step-9: Stop

VI Insertion in Sorted List:

Step-1: If avail = NULL then write overflow & stop

Step-2: $Ptr = avail$
 $avail = Rpt(avail)$
Read info (Ptr)

Step-3: $Cpt = first$

Step-4: Repeat Step-5 while
 $Info(Cpt) < Info(ptr)$

Step-5: $Cpt = Rpt(Cpt)$

Step-6: $Tpt = Lpt(Cpt)$

Step-7: $Rpt(Tpt) = Cpt$
 $Lpt(Cpt) = Tpt$
 $Rpt(Tpt) = Ptr$
 $Lpt(Ptr) = Tpt$

Step-8: Stop

Program:

```
void IGLC()
{
    int data;
    Ptr = (struct node*) malloc (size of (struct node));
    printf("Enter info of node");
    scanf("%d", &ptr->info);
    printf("Enter Data");
    scanf("%d", &Data);
    Cpt = first;
```



```

while (Cpt → info != data)
{
    Cpt = Cpt → Rpt;
    Tpt = Cpt → Lpt;
    Pte → Rpt = Cpt;
    Cpt → Lpt = Pte;
    Tpt → Rpt = Pte;
    Pte → Lpt = Tpt;
}

```

VII Deletion of first node:

Step-1: If first = NULL then write underflow & stop.

Step-2: Pte = first

Step-3: first = Rpt(Pte)
Lpt(first) = NULL

Step-4: Rpt(Pte) = Avail
Avail = Pte

Step-5: Stop

VIII Deletion from end:

Step-1: If first = NULL then write underflow & stop.

Step-2: Pte = first

Step-3: Repeat Step-4 while Rpt(Pte) != NULL

Step-4: Pte = Rpt(Pte)

Step-5: Cpt = Lpt(Pte)

Step-6: Rpt(Cpt) = NULL

Step-7: Rpt(Pte) = Avail
Avail = Pte

Step-8: Stop

IX Deletion of Particular Node:

Step-1: If first = NULL then write underflow & stop.

Step-2: Read data as info of node

Step-3: Pte = first

Step-4: Repeat Step-5 while info(Pte) != data

Step-5: Pte = Rpt(Pte)

Step-6: Cpt = Lpt(Pte)
Tpt = Rpt(Pte)

Step-7: Rpt(Cpt) = Tpt
Lpt(Tpt) = Cpt

Step-8: Rpt(Pte) = Avail

avail = Ptr

Step-9: Stop

19/8/19 Stack:

4	Ex	Top
3	Dx	
2	C	
1	B	Top
0	A	

Top = -1

Push → Insertion of element
Pop → Deletion of element

A stack is an ordered collection of homogeneous data elements where the insertion & deletion occur at only one end. This end is known as the top of the stack.

In stack last element will be deleted first so, it is called the last in first out (LIFO).

Push - The process of adding a new element to the top of the stack is called the push operation.

Pop - The process of deleting an element from the top of the stack is called the pop operation.

Algo for Push:

Step-1: If $top = Max - 1$ then write overflow & stop.

Step-2: Read Data

Step-3: $Top = Top + 1$

Step-4: $Stack[Top] = Data$

Step-5: Stop

Algo for pop:

Step-1: If $top < 0$ then write underflow & stop

Step-2: $Stack[Top] = NULL$

Step-3: $Top = Top - 1$

Step-4: Stop

Q1. WAP to implement the push & pop operation on a stack with the help of array.

Q2. WAP to implement the push & pop operation on a stack with the help of linked list.

Ans2. struct node

```
{ int Info;  
  struct node * Link;  
};
```

```
struct node * Ptr, * Cpt, * Top;  
void main()
```



```

{ char ch;
  Ptr = (struct node *) malloc (size of (struct node));
  printf ("Enter first node info ");
  scanf ("%d", &ptr->Info);
  ptr->Link = NULL;
  do
  { Cpt = (struct node *) malloc (size of (struct node));
    printf ("Enter next node info ");
    scanf ("%d", &Cpt->Info);
    Cpt->Link = Ptr;
    Ptr = Cpt;
    printf ("Do you want to add more nodes (Y/N)");
    scanf ("%c", &ch);
  } while (ch == 'Y' || ch == 'y');
  Top = Ptr;
  getch();
}

```

Polish Notation :

Transforming an expression from one form to another form is called the polish notation. An arithmetic expression can be represented in various forms such as postfix, prefix, infix.

The prefix, infix & postfix refers to the relative position of an operator with respect to its operand. If the operator is placed before its operand it is called the prefix. If the operator is placed after its operand it is called postfix. If the operator is placed b/w two operands then it is called infix.

Infix - $A+B$
Prefix - $+AB$
Postfix - $AB+$

Priority: Exponential $\uparrow$
*, /
+, -

* Convert Infix to Postfix:

eg. $(A+B) / (C-D)$
 $(AB+) / (CD-)$
 $AB+CD-/$

(6) $A + (B * C - (D / E \wedge F) * G) * H$

$A + (B * C - (D / E \wedge F) * G) * H$
 $A + (B * C - (D E \wedge F /) * G) * H$
 $A + (B * C - (D E \wedge F / G *)) * H$
 $A + (B C * - D E \wedge F / G *) * H$
 $A + (B C * D E \wedge F / G * -) * H$
 $A + (B C * D E \wedge F / G * - H *)$
 $ABC * D E \wedge F / G * - H * +$

* Infix to Postfix using Stack:

Algorithm:

Step-1: Scan the infix expression from left to right.

Step-2: If scan character is an operand then it will display into the postfix expression.

Step-3: If scan character is an operator then it has the greater priority than stack operator

then it will also push into the stack

Step-4: If scan character is less than or equal to stack operator then scan character will pop out from stack & push into the postfix notation.

Step-5: If scan character is right parenthesis then till left parenthesis of stack all characters will pop out & left parenthesis will be cancelled by right parenthesis.

Step-6: Repeat all steps until the stack is empty.

Eg: $5 * (6 + 2) - (12 / 4)$

Char No.	Scanned	Stack	Postfix Expression
1.	5		5
2.	*	*	5
3.	(	*(	5
4.	6	*(	5, 6
5.	+	*(+	5, 6
6.	2	*(+	5, 6, 2
7.	)	*	5, 6, 2, +
8.	-	-	5, 6, 2, +, *
9.	(	-(	5, 6, 2, +, *
10.	12	-(	5, 6, 2, +, *, 12
11.	/	-(/	5, 6, 2, +, *, 12
12.	4	-(/	5, 6, 2, +, *, 12, 4
13.	)	-	5, 6, 2, +, *, 12, 4, /

Q3: Convert $A + (B * C - (D / E * F) * G) * H$ expression into postfix with the help of stack

Ans.	S.No.	Scanned Char	Stack	Postfix Expression
	1.	A		A
	2.	+	+	A
	3.	(	+(	A,
	4.	B	+(	A, B
	5.	*	+(*	A, B
	6.	C	+(C*	A, B, C
	7.	-	+(C-	A, B, C, *
	8.	(	+(C-(	A, B, C, *
	9.	D	+(C-(	A, B, C, *, D
	10.	/	+(C-(/	A, B, C, *, D
	11.	E	+(C-(/	A, B, C, *, D, E
	12.	*	+(C-(/	A, B, C, *, D, E
	13.	F	+(C-(/	A, B, C, *, D, E, F
	14.	)	+(C-	A, B, C, *, D, E, F, A, /
	15.	*	+(C-*	A, B, C, *, D, E, F, A, /
	16.	G	+(C-*	A, B, C, *, D, E, F, A, /, G
	17.	)	+	A, B, C, *, D, E, F, A, /, G, *, -
	18.	*	+	A, B, C, *, D, E, F, A, /, G, *, -
	19.	H	+	A, B, C, *, D, E, F, A, /, G, *, -H

Ans = $ABC * DEF / G * - H * +$

22.8.19 * Convert Infix to Prefix:

Eg: (a) $A / B * C - D$
 $A / \wedge B * C - D$
 $/ \wedge A B C - D$

$$= - / A A B C D$$

Eg: (b) $(A-B/C) + (D * E - F)$
 $(A - / BC) + (* DE - F)$
 $(- A / BC) + (- * DEF)$
 $+ - A / BC - * DEF$

* Infix to ^{Prefix} Using Stack:

Algorithm:

Step-1: Reverse the input string

Step-2: Examine the next element in the input.

Step-3: If it is operand, then add it to output string.

Step-4: If it is closing parenthesis, add it to stack.

Step-5: If it is an operator:

(a) If stack is empty push operator in the stack

(b) If the top of stack is closing parenthesis, then also push the operator in stack

(c) If it has the same or greater priority, then push operator to stack

(d) Else pop the operator & add to output string.

Step-6: If it is opening parenthesis, pop the operator from stack and add them to output string until closing parenthesis occurred.

Step-7: If there is no input, then go to Step-2.

Step-8: If there is no more input, then un-step the remaining operator, & add them to op stage.

Step-9: Reverse the output string.

Eg: $2 * 3 / (2 - 1) + 5 * (4 - 1)$
 Reverse string $\rightarrow) 1 - 4 (* 5 +) 1 - 2 (/ 3 * 2$

SNo.	Scanned Char	Stack	Prefix Expression
1.	)	)	
2.	1	)	1
3.	-	) -	1
4.	4	) -	1, 4
5.	(		1, 4, -
6.	*	* -	1, 4, -
7.	5	*	1, 4, -, 5
8.	+	+	1, 4, -, 5, *
9.	)	+	1, 4, -, 5, *
10.	1	+	1, 4, -, 5, *, 1
11.	-	+) -	1, 4, -, 5, *, 1
12.	2	+) -	1, 4, -, 5, *, 1, 2
13.	(	+	1, 4, -, 5, *, 1, 2, -
14.	/	+/	1, 4, -, 5, *, 1, 2, -
15.	3	+/	1, 4, -, 5, *, 1, 2, -, 3
16.	*	+/ *	1, 4, -, 5, *, 1, 2, -, 3
17.	2	+/ *	1, 4, -, 5, *, 1, 2, -, 3, 2

$$\text{Ans} = 14 - 5 * 12 - 32 * / +$$

Algorithm to evaluate the postfix expression using Stack:

Step-1: If an operand is encountered, then push it into stack.

Step-2: If an operator is encountered, then
(a) Pop two element from the stack, where A is the top of element & B is the next top element.
(b) Evaluate B operator A
(c) Push the result on stack.

Step-3: The evaluated value is equal to the value at the top of stack.

$$\text{Eg: } 5, 6, 2, +, *, 12, 4, /, -$$

SNo	Scanned Char	Stack	Operation (B op A)
1.	5	5	
2.	6	5, 6	
3.	2	5, 6, 2	
4.	+	5, 8	$6 + 2 = 8$
5.	*	40	$5 * 8 = 40$
6.	12	40, 12	
7.	4	40, 12, 4	
8.	/	40, 3	$12 / 4 = 3$
9.	-	37	$40 - 3 = 37$
		$\Rightarrow \text{Ans} = 37$	

$$\text{Eg: } 5, 9, 8, +, 4, 6, *, +, 7, -, *$$

SNo	Scanned Char	Stack	Operation (B op A)
1.	5	5	
2.	9	5, 9	
3.	8	5, 9, 8	
4.	+	5, 17	$9 + 8 = 17$
5.	4	5, 17, 4	
6.	6	5, 17, 4, 6	
7.	*	5, 17, 24	$4 * 6 = 24$
8.	+	5, 41	$17 + 24 = 41$
9.	7	5, 41, 7	
10.	-	5, 34	$41 - 7 = 34$
11.	*	170	$5 * 34 = 170$

$$\text{Ans} = 170$$

Checking the Validity of an Arithmetic Expression using Stack:

Step-1: Whenever an opening parenthesis encountered push it into the stack.

Step-2: Whenever a closing parenthesis is encountered, the stack is examine.

Step-3: If the stack is empty, the closing parenthesis doesn't have any opening parenthesis then the expression is invalid.

Step-4: If the stack is not empty, pop the stack and check whether the pop item corresponds to

the closing paranthesis.

Step 5: If the match occurs, then we will continue, otherwise the expression is invalid.

Step 6: When the end of expression is reached the stack must be empty.

Eg: $[(A+B) - \{C+D\}] - [F+G]$

S. No	Symbol Scanned	Stack
1.	[	[
2.	(	[(
3.	A	[(
4.	+	[(
5.	B	[(
6.	)	[
7.	-	[-
8.	{	[-{
9.	C	[-{
10.	+	[-{
11.	D	[-{
12.	}	[-
13.	]	empty
14.	-	[-
15.	[	[
16.	F	[
17.	+	[
18.	G	[
19.	]	empty

Since the stack is empty at end, hence it is valid.

()

Queue:

In a queue, there are two ends front & rear.

front $\rightarrow$ to delete element from queue
rear $\rightarrow$ to insert element to queue.

front = rear = 0 (empty queue)

Inserting $\rightarrow$

A	B	C	D		
---	---	---	---	--	--

 Front Rear

As we insert the element value the value of rear increases.

Deleting $\rightarrow$

	B	C	D		
--	---	---	---	--	--

 Front Rear

As we delete the element value the value of front increases.

A queue is a linear list of elements in which deletion can take place only at one end which is from the front & insertion can take place only at the other end which is from the rear. Therefore, queue follows the FIFO operation i.e. first in first out operation.

FIFO $\rightarrow$

A	B	C	D	E	
---	---	---	---	---	--

 Front Rear

~~There are~~ There are only two operation possible with the queue:

- (i) Push
- (ii) Pop

Algorithm:

(i) Push:

Step-1: If $Rear = \max$, then write overflow & stop

Step-2: Read data

Step-3: If $front = 0$; then
(a) $front = Rear = 1$
(b) $Q[Rear] = Data$
else
(a) $Rear = Rear + 1$
(b) $Q[Rear] = Data$

Step-4: Stop

(ii) Pop:

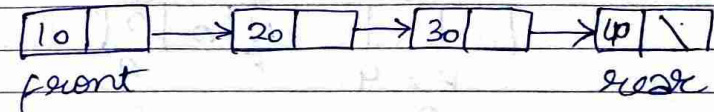
Step-1: If $front = 0$ then write under flow & stop.

Step-2: If $front = Rear$ then
(a) $Q[front] = NULL$
(b) $front = 0$

else
(a) $Q[front] = NULL$
 $front = front + 1$

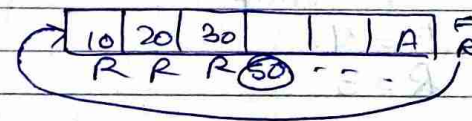
Step-3: Stop

Linked Representation of Queue:

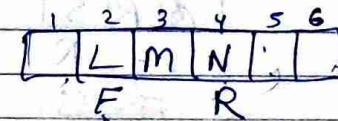


⇒ Insertion at end & for deletion we use deletion from begin.

Circular Queue:

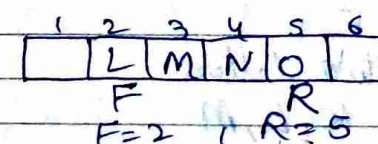


Q1. A circular queue capable of accommodating max. 6 elements, $Rear = 4$, $Front = 2$ & $Q = LMN---$

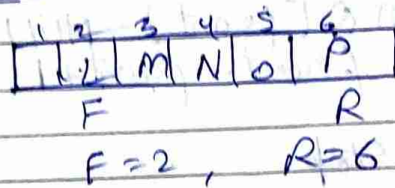


- (a) Insert O.
- (b) Insert P
- (c) Delete two letters
- (d) Insert Q, R, S
- (e) Delete 1 letter

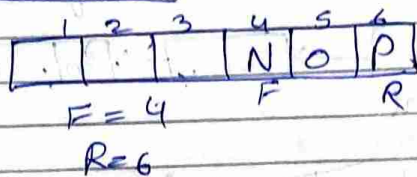
Ans(a) Insert O:



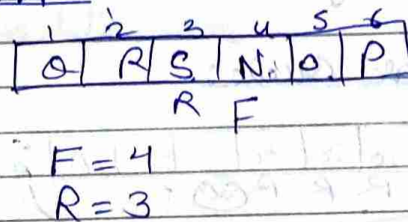
(b) Insert P:



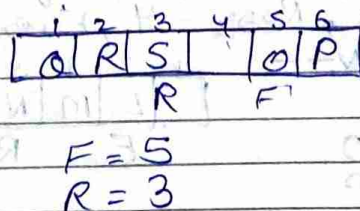
(c) Delete two Letters:



(d) Insert O, R, S:



(e) Delete 1 letter:



Q. Circular queue of 6 memory cell, $F=2, R=4$ & the queue is - ACD -

(a) Insert F

(b) Delete two letters

(c) Insert K, L, M

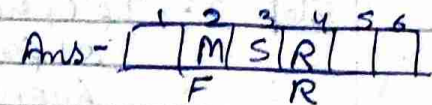
(d) Delete two letters

(e) Insert S

(f) Delete 1 Letter

(g) Insert R

(h) Delete 1 letter



Algorithms for Circular Queue:

(i) Push:

Step-1: If $(\text{front} = 1 \& \text{Rear} = \text{Max}) \vee (\text{front} = \text{Rear} + 1)$ then write overflow & stop.

Step-2: Read Data

Step-3: If $(\text{front} = 0)$
 $\text{front} = \text{Rear} = 1$

Step-4: If $(\text{Rear} = \text{Max})$, then
 $\text{Rear} = 1$
else
 $\text{Rear} = \text{Rear} + 1$

Step-5: $\text{co}[\text{Rear}] = \text{Data}$

Step-6: Stop

(ii) Pop:

Step-1: If $(\text{front} = 0)$, then underflow & stop.

Step-2: If $\text{front} = \text{Rear}$, then

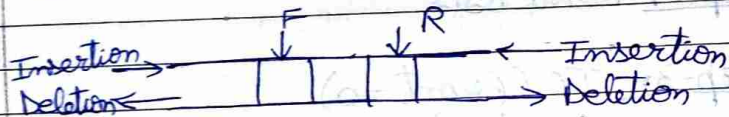
$ca[front] = NULL$
 $front = rear = 0$

Step-3: If $(front = Max)$, then
 $ca[front] = NULL$
 $front = 1$
else
 $ca[front] = NULL$
 $front = front + 1$

Step-4: Stop

26.8.19

Double Ended Queue (DEQUEUE):



Front $\rightarrow$ left
Rear $\rightarrow$ Right

In Dequeue both insertion & deletion operations are performed at either end of the queue that means we can insert an element from rear end or the front end & also deletion is possible from the front end and the rear end.

The most common ways of representing a DEQUEUE are:

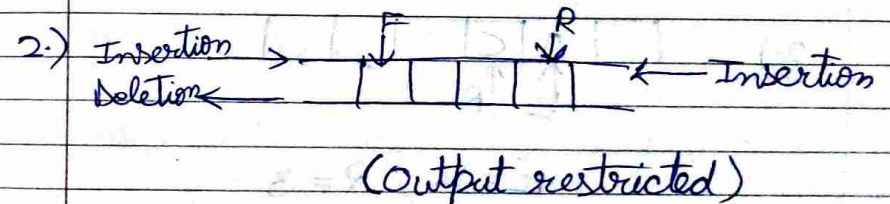
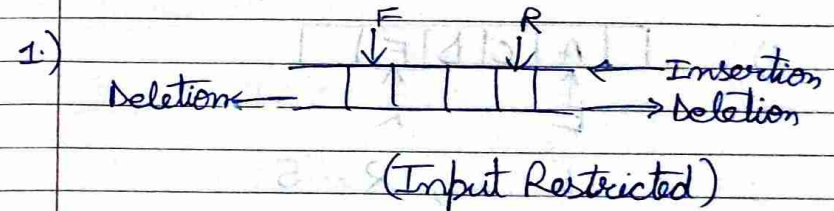
- 1) Doubly Linked List
- 2) Circular array

Types of Dequeue:

- 1) Input restricted Dequeue
- 2) Output restricted Dequeue

* In input restricted Dequeue the element can be inserted only at one end but we can delete the element from both end.

* In output restricted Dequeue the deletion can be performed only from one end but the insertion is possible from both the ends.

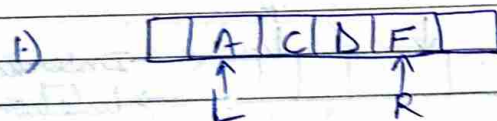
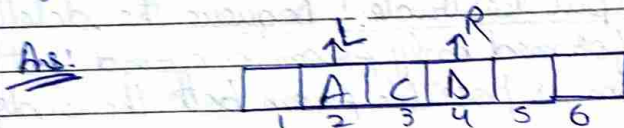


Possible Operations Performed with DEQUEUE:

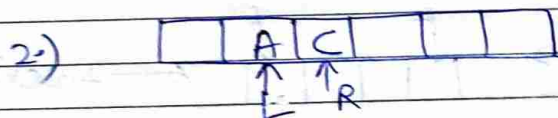
- 1) Insertion of an element from Rear end.
- 2) Insertion of an element from front end.
- 3) Deletion of an element from front end.
- 4) Deletion of an element from rear end.

Eg: Left = 2, Right = 4
DEQUEUE: — A C D —

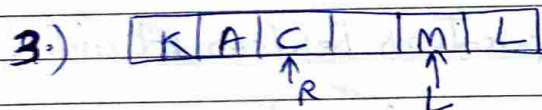
- 1) Insertion of F from Right
- 2) Delete 2 letters from Right
- 3) K, L, M are inserted from left
- 4) One letter is deleted from left
- 5) Insert R from right
- 6) Insert S from left



L = 2, R = 5

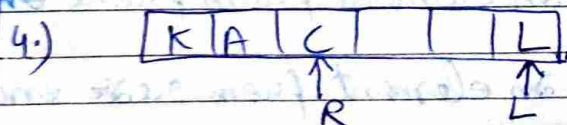


L = 2, R = 3

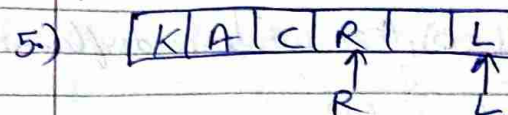


L = 5, R = 3

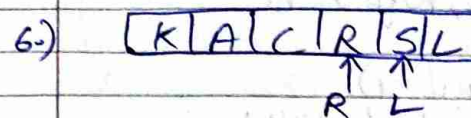
(If there is no space left from front to insert the letter then we start from rear end)



L = 6, R = 3



L = 6, R = 4



L = 5, R = 4

Insertion From Left:

Step-1: If (Left = 1 & Right == Max)
 !! (Left = Right + 1),
 then write overflow & stop.

Step-2: Read Data

Step-3: If (Left = 0)

Left = 1

Right = 1

else

If (Left = 1)

Left = 0

Right = 0

else

Left = Left - 1

Step-4: Data[Left] = Data

Step-5: Stop

Step 5: Step Deletion from Right:

Step-1: If (Left = 0), then write underflow & stop

Step-2: If (Left = Right)
DO[Right] = NULL
Left = 0, Right = 0.
else
if (Right = 1)
DO[Right] = NULL
Right = Max
else
DO[Right] = NULL
Right = Right - 1

Step-3: Stop

Priority Queue: Priority queue is a data structure in which each element has a assigned value called the priority of element and the element can be inserted or deleted not only at the end but at any position in the Queue ~~or~~ to its priority.

Basic Properties of Priority Queue:

1. Every element has a priority associated with it. ~~is~~ is dequeued
2. An element of higher priority ~~is processed~~ ^{before an} element of lower priority.
3. Two elements with the same priority are processed ⁱⁿ the order in which they were inserted to the Queue.

4. These rules are followed by the Queue in case of deletion

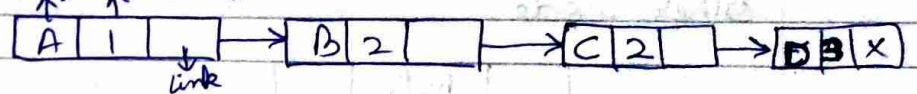
There are two types of priority queue:

- (a) Ascending Priority Queue
- (b) Descending Priority Queue

(a) Ascending Priority Queue: In an ascending priority queue elements can be inserted in an order but while deleting the element always a small element ^{will} ~~only~~ be deleted. _{First}

(b) Descending Priority Queue: But in descending priority queue while deleting the element, ^{always} largest element will be deleted first.

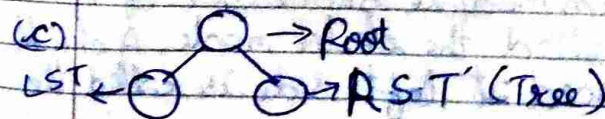
Info ↑ Priority No. ↓

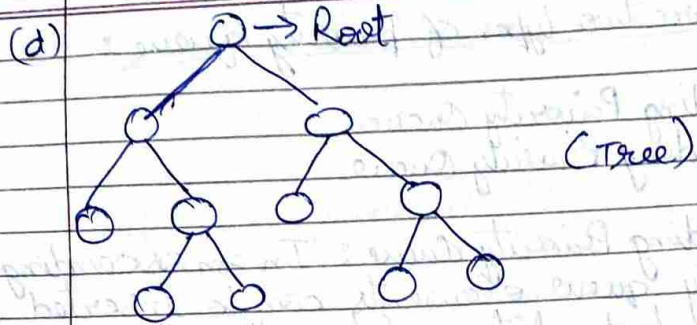


Trees: A tree is either empty or it consists of a node called the root together with two binary tree called the left sub-tree & right sub-tree of the root. The definition of a tree doesn't ^{impose} ~~impose~~ the number of children of given node. The number of children ~~can~~ can vary from 0 to any integer.

(a) Blank Tree

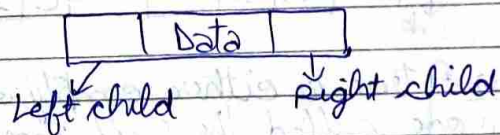
(b) $\bigcirc \rightarrow \text{Root}$





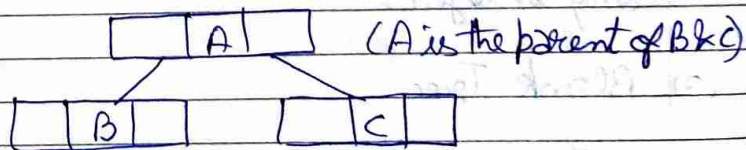
Terminology used with Tree:

1. Node: We will use the term node rather than ~~vertex~~ vertex with tree. This is the main component of any tree structure. It stores the actual data along with link of other node.



Left & Right child are pointers of structure Node.

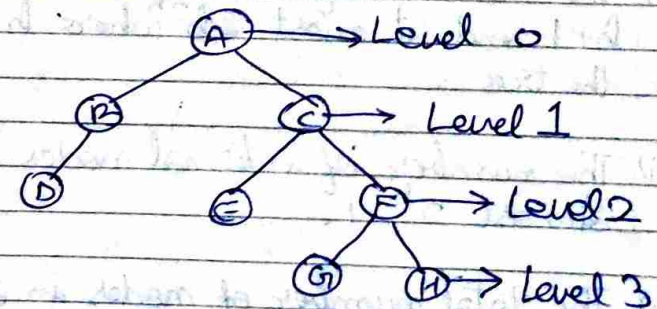
2. Parent: The parent of the node is the immediate predecessor of that node.



3. Child: The immediate successor of a node ~~are~~ called the child node. A child which is placed at left side is called the

left child & vice-versa.

4. Root: This is the node which has no any parent. In the complete tree there is only one root.
5. Leaf Node (External Node): A node which has no child is called the leaf node.
6. Siblings: The child nodes of a given parent node ~~are~~ called the siblings.
7. Branch or Edge: The connecting line b/w two nodes is called the branch or edge. A node can have more than one edge.
8. Path: Each node has to be reachable from the root through a unique sequence of edges called the path. The number of edges in a path are called the ~~length~~ length of path.
9. Level of Node: The level of the node is the number of edges along the unique path b/w it and the root. The level of the root is defined as 0.



10. Height of the tree: The height of the tree is the maximum level of the node + 1.

$$i.e. [h = I_{max} + 1]$$

↓
maximum level

23.8.19 # Binary Tree: A tree is a binary tree if & only if,

- (i) It has a root node which may not have any child node (0 child, NULL Tree).
- (ii) Root node may have 1 or 2 children nodes & each such node forms a binary tree itself.
- (iii) Number of child nodes can be 0, 1, 2 but not more than 2.
- (iv) There is a unique path from the root to every other node.

Properties of Binary Tree:-

- (i) The number of external nodes is 1 more than the number of internal nodes.
- (ii) The number of external nodes are at least $h+1$ and at most 2^h where h is the height of the tree.
- (iii) The number of internal nodes is at least h and at most $2^h - 1$.
- (iv) The total number of nodes in a binary tree is

at least ~~2~~ and at most $2^{h+1} - 1$.

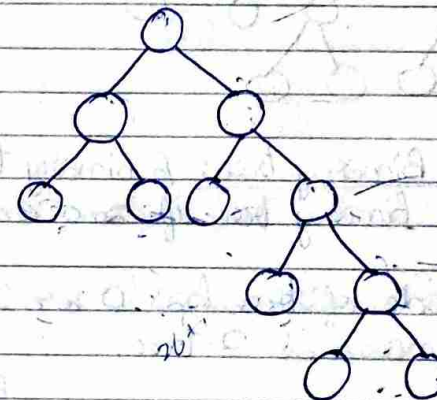
- (v) The height h of a binary tree with n number of nodes is at least $\log(n+1)$ and at most n .
- (vi) A binary tree with n nodes has exactly $n-1$ edges

Types of Binary Tree:

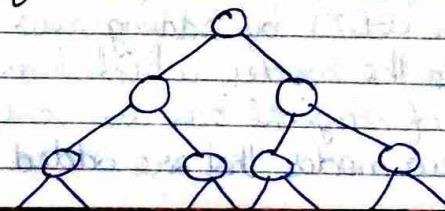
- (a) Strictly Binary Tree: A binary tree is a strictly binary tree if and only if each node has exactly 2 child node or 0 child node.

$$n+1 = m$$

$$m-1 = n$$



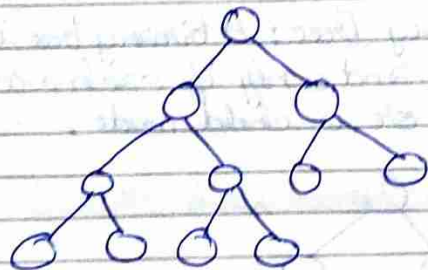
- (b) Full Binary Tree: A binary tree is a full binary tree if & only if,
 - (i) Each non leaf node has exactly 2 child node.
 - (ii) All leaf node are at same level.



(c) Complete Binary Tree: A binary tree is a complete binary tree if and only if,

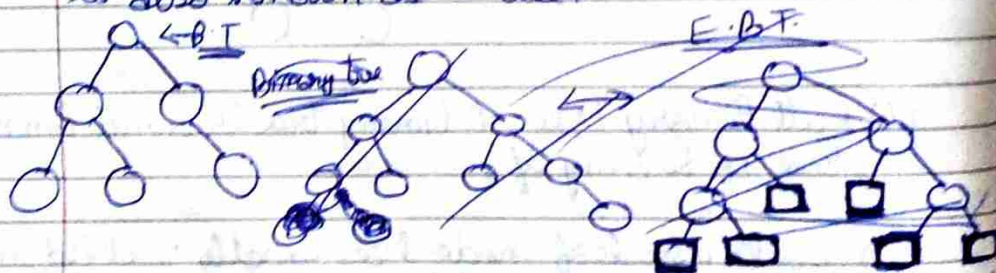
(i) Level 0 to $h-1$ represents a full binary tree of height $h-1$.

(ii) One or more nodes in level $h-1$ may have 0 or 2 child nodes.



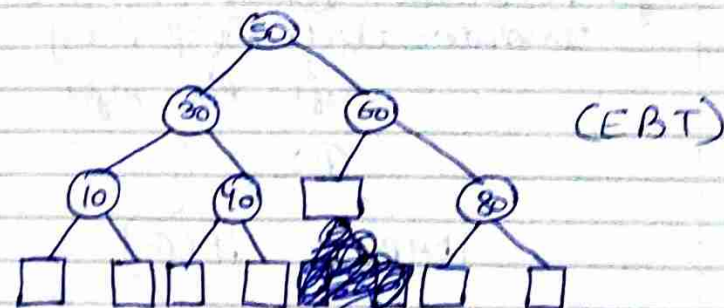
(d) Extended Binary tree: A binary tree is called an extended binary tree if and only if,

(i) Every node of tree has 0 or 2 children. It is also known as 2-tree.



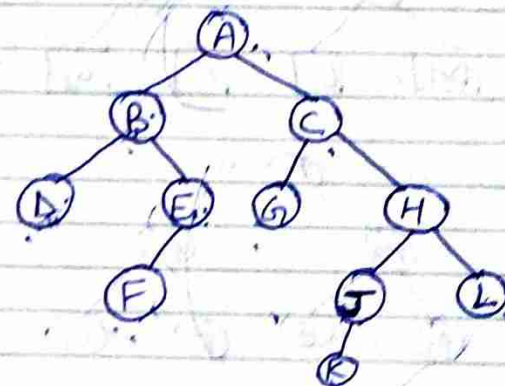
→ A binary tree can be converted to an Extended Binary Tree (EBT) by adding new node to its left node, & to the nodes which have only one child. The nodes of original tree are called internal nodes and the new nodes that are added to binary tree, are

called external nodes. It is very useful to represent algebraic expression.



Traversal of Tree:

Eg:



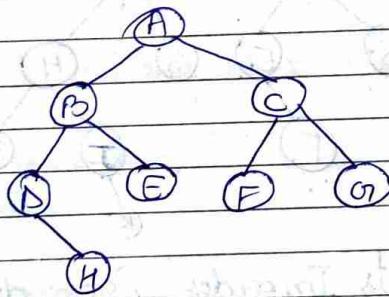
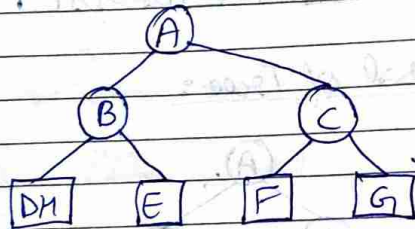
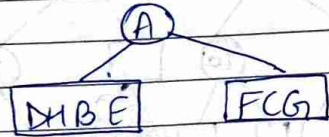
Write its Inorder, Preorder & Postorder.

- (i) For inorder: Left → Root → Right
- (ii) For Preorder: Root → Left → Right
- (iii) For Postorder: Left → Right → Root

Ans. Inorder: D B F E A G C K J H L
Preorder: A B D E F C G H J K L
Postorder: D F E B G K J L H C A

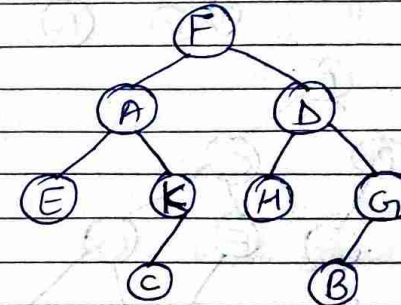
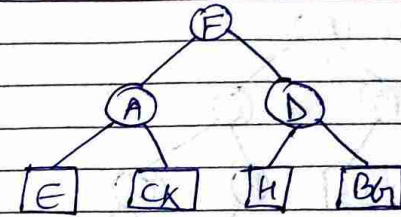
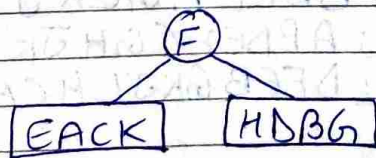
Creation of Binary Tree:

Eg: Preorder: ^{Root} A B D H E C F G
Inorder: D H B E A F C G
 left Root Right

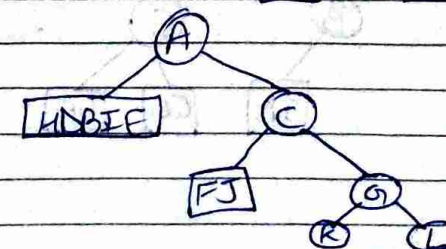
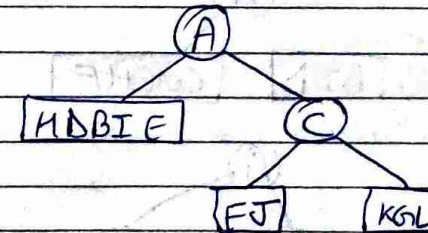
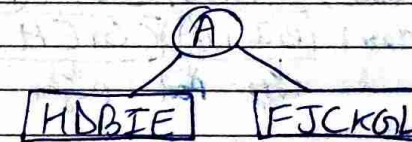


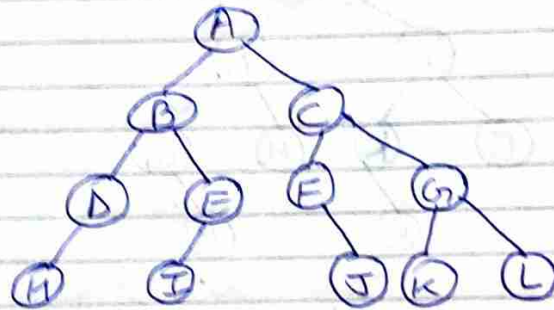
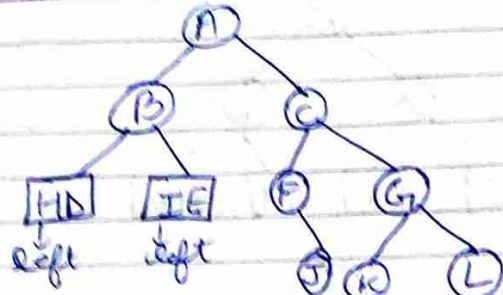
⇒ Postorder: H D E B F G C A

Eg: Preorder: ^{Root} F A E K C D H G B
Inorder: E A C K F H D B G
 left Root Right



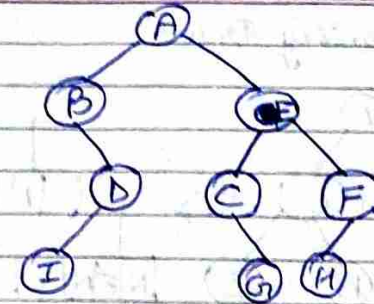
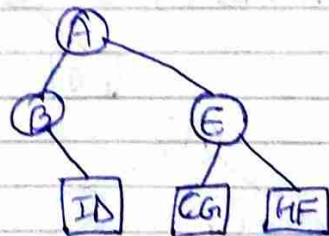
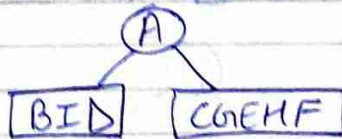
Eg: Postorder: H D E B F K L G C A → Root
Inorder: ~~H~~ A B I E A F J C K G L
 left Root Right





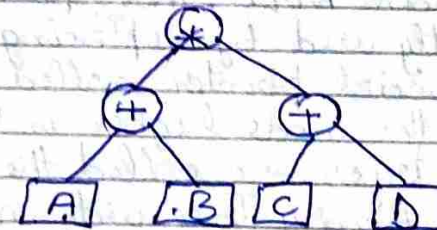
Preorder: A B D H E I C F J G K L

Eg. Postorder: I D B G J C H F E A → Root
Inorder: B I H A C G E H F
 ↓ ↓ ↓
 left Root Right

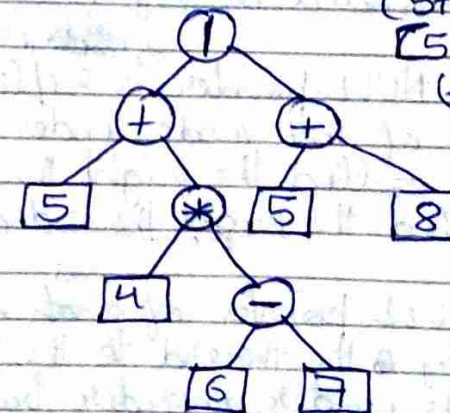


Expression Tree:

Eg: $(A+B) * (C+D) = (+AB) * (+CD)$
Preorder: $* + AB + CD$
operator → ○
operand → □

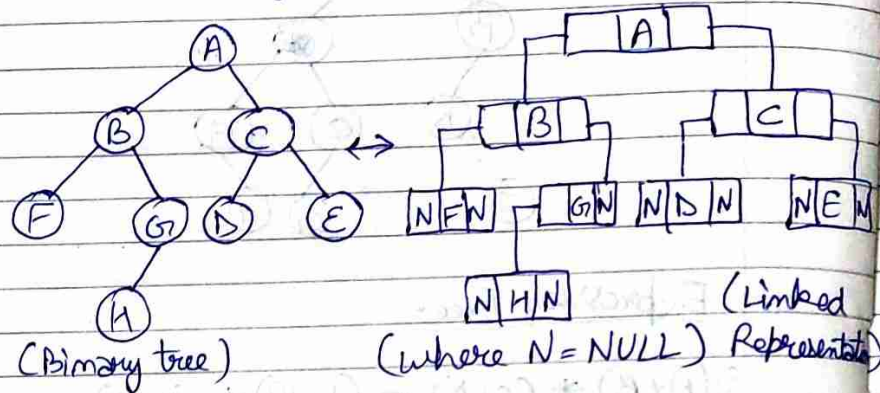


Eg: $(5+4 * (6-7)) / (5+8)$



$(5+4 * (6-7)) / (5+8)$
 $[5 + (* 4 - 67)] / (+58)$
 $(+5 * 4 - 67) / (+58)$
 $/ + 5 * 4 - 67 + 58$

Threaded Binary Tree:



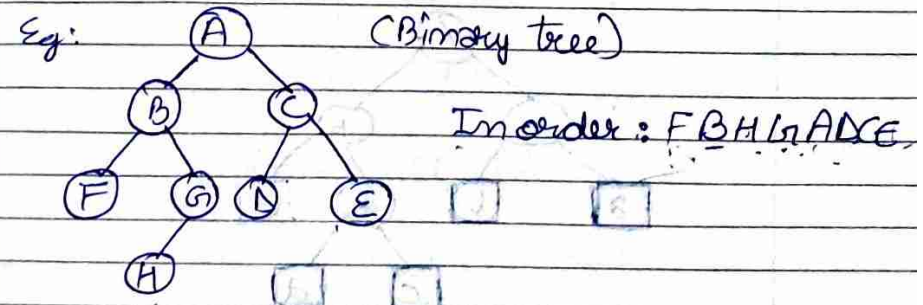
If we consider the linked representation of any binary tree, then we noticed that half of the entries in the pointer field left & right will contain NULL. This space may be more efficiently used by replacing NULL entries by the special pointer called the thread, which points to the node higher in the tree. These type of trees are called the Threaded Binary Tree. A Binary Tree with n no. of nodes where $n > 0$ has exactly $n+1$ NULL links.

Types of Threaded Binary Tree:

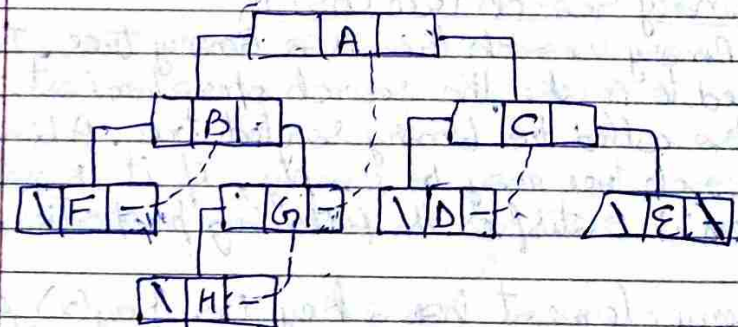
- The right NULL pointer, ^{of each node} is replaced by the successor of that node under inorder traversal called the right thread & the tree is called the right threaded tree.
- The left NULL pointer of ^{each} node can be replaced by the thread to the predecessor of that node under inorder traversal called

the left threaded & the tree is called left threaded binary tree.

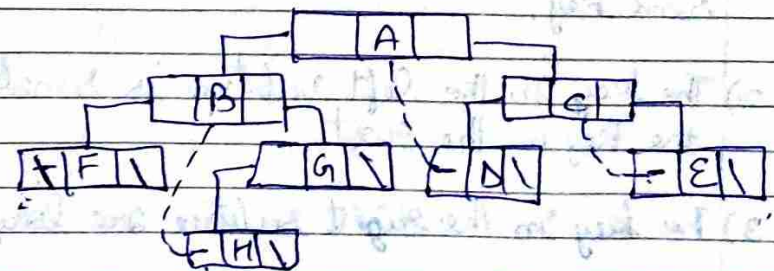
- Both left & right NULL pointers can be used to point to the predecessor & successor of the node respectively under inorder traversal. Such a tree is called the Fully threaded binary tree.



Right threaded tree:



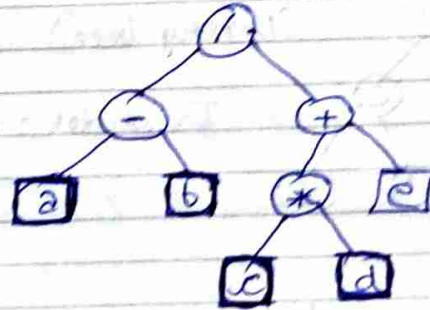
Left threaded tree:



24/9/19

Q1. Create an expression tree for $(a-b)/(C*d)+e$

Ans: Inorder = $(a-b)/(C*d)+e$
 Preorder = $(-ab)/(*cd)+e$
 $= -ab/+*cde$
 $= [-ab+*cde]$



Binary Search Tree (BST):

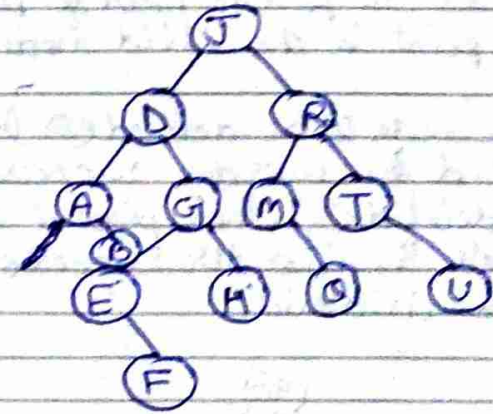
A Binary search tree is a binary tree. It is used to make the search operation fast. It is also called the binary sorted tree. A binary search tree may be empty, if it is not empty then it satisfies the following properties,

- 1) Every element has a key i.e. Key(x) for element x and no two element can have the same key.
- 2) The key in the left subtree is smaller than the key in the root.
- 3) The key in the right subtree are larger than the

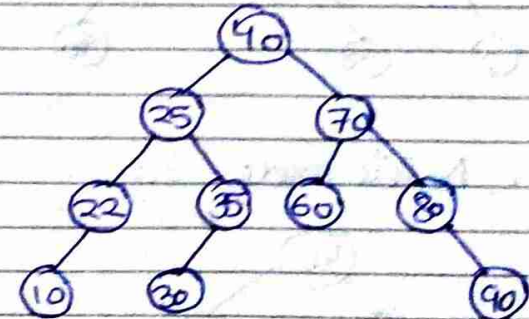
key ~~is~~ in the root.

4) The left & right subtree are also the binary search tree.

Eg: J R D T G E A M H F O U B
 Create the Binary search tree



Eg: 40, 25, 70, 22, 35, 60, 80, 90, 10, 30
 Create the Binary Search tree.



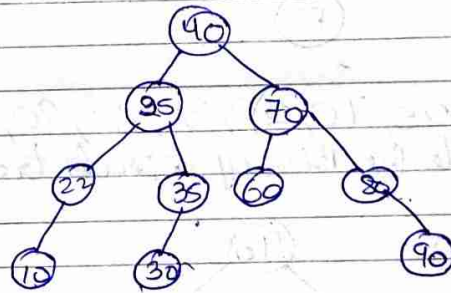
Deletion of a node in Binary Search Tree (BST):

Deletion of a node in BST depends on its children.

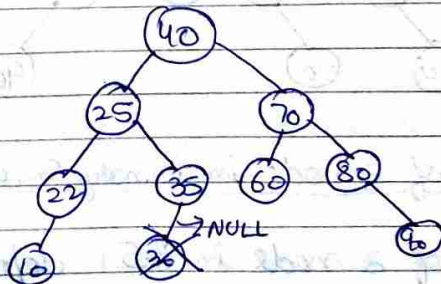
There are total 3 cases:

- (i) If the node to be deleted (2) is a leaf node; simply make the pointer field of parent node of (2) to NULL.
- (ii) ^{If} the node to be deleted (2) has a child; then simply set the pointer field of parent node of (2) to point to the child node of (2).
- (iii) If the node to be deleted (2) has 2 child; then find the inorder successor of the node to be deleted (2), then place it in the node (2) & delete the inorder successor node similar to case (i) or (ii).

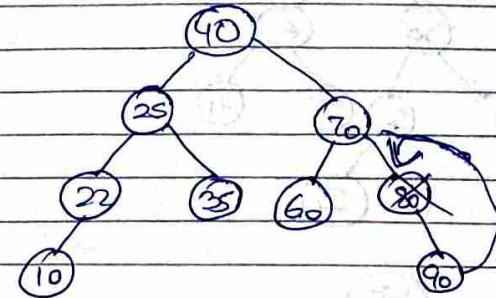
Eg:



Case-I, Delete node - 30

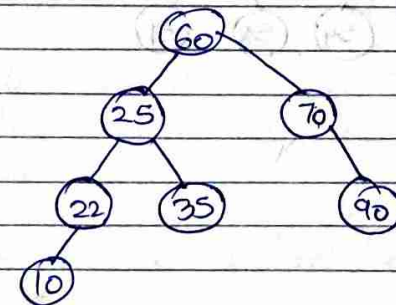


Case-II Delete node - 80

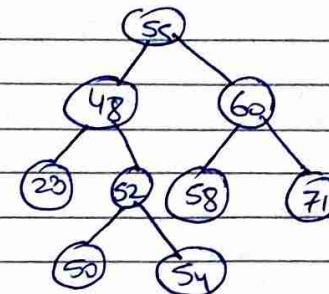


Case-III: Delete node - 40

In order - 10, 22, 25, 35, 40, 60, 70, 90
(Replace 40 by 60)



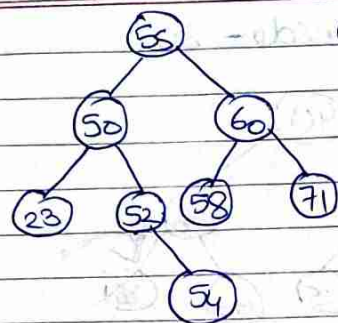
Eg:



Delete node 48 & 52

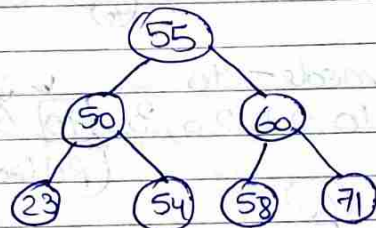
In 23, 48, 50, 52, 54, 55, 58, 60, 71 (Inorder)

Replace 48 by 50



(After deleting node 43)

(ii)



(After deleting node 52)

3.10.19

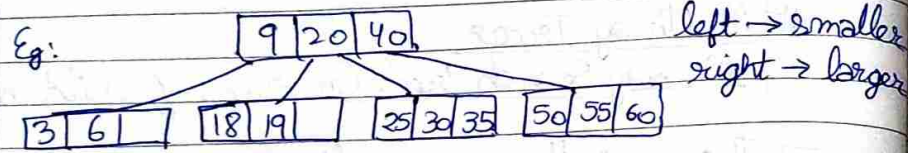
Unit-III

Multway Trees

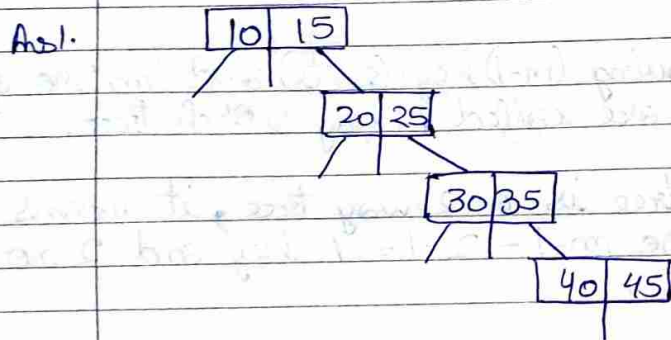
M-way Search tree: ($m \rightarrow$ no. of child nodes/paths)

- They are the generalise version of binary search tree. The goal of M-way search tree is to minimize the accesses while retrieving a key from a file.
- Trees having $(m-1)$ keys (roots) and 'm' no. of children are called M-way search tree.
- A binary tree is a 2-way tree, it means it has the $m-1 = 2-1 = 1$ key and 2 no. of children.
- The binary tree is called the M-way tree of order 2.
- There are some properties of M-way trees:
 - (i) In this tree, all the nodes have the degree $\leq m$.
 - (ii) The keys in each node are in the ascending order.
 - (iii) The key K_i is larger than the key in subtree pointed by P_i and smaller than the key in subtree ~~po~~ pointed by P_{i+1} .

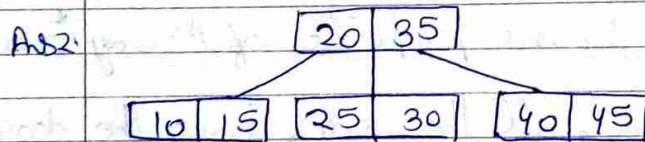
(iv) The subtrees are also the M-way tree.



Q1. Create a 3-way search tree
10, 15, 20, 25, 30, 35, 40, 45



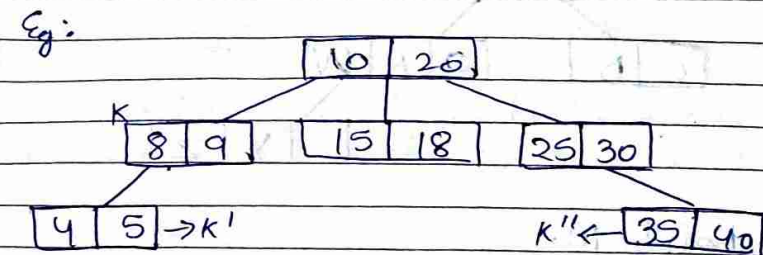
Q2. Create a tree of order 3.
20, 35, 40, 10, 15, 25, 30, 45



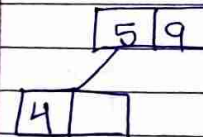
Deletion in M-Way Search tree:

- (i) If $A_i = A_j = \text{NULL}$, then delete K .
- (ii) If $A_i \neq \text{NULL}$, $A_j = \text{NULL}$, then choose the largest of key element K' in the child node and replace K by K' .
- (iii) If $A_i = \text{NULL}$ and $A_j \neq \text{NULL}$, then choose the smallest of key element K'' from subtree

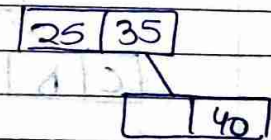
and replace ~~any~~ K by K'' .



Case - (ii)



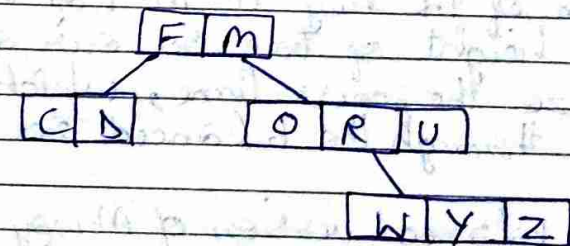
Case - (iii)



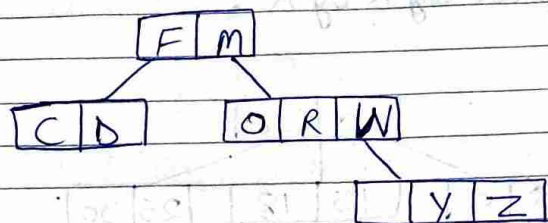
(iv) If $A_i \neq \text{NULL}$ & $A_j \neq \text{NULL}$, then choose either the greatest or smallest and replace.

Q1. Create a M-way search tree.
10, 15, 20, 25, 30, 35, 40, 45

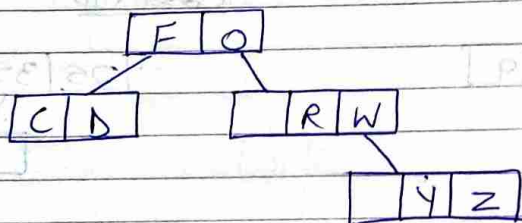
Q2. Delete U and M from the given tree.



Ans2. Delete U:
Right subtree $\rightarrow K'' = \text{smallest} = W$



Delete M:
 $K'' = 0$



B-Tree (Balanced Version of M-way search tree)

In M-way search tree so many nodes have the left subtree but no right subtree similarly, so many nodes have right subtree but no left subtrees. Now, insertion of the key in M-way tree as increase the height of tree. So, our aim is to minimize the access time, which can be possible through the balanced tree only.

The balanced version of M-way search tree is called the B-tree. So, the B-tree of order 'M' has the following properties.

1. All leaf nodes are at same level.
2. It follows bottom-up process i.e. insertion takes place from bottom to top.

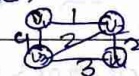
3. Every non-leaf node (except root) has at least $\lceil m/2 \rceil$ child nodes.

14.10.19

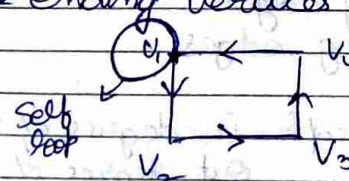
Graph:

Terminology:

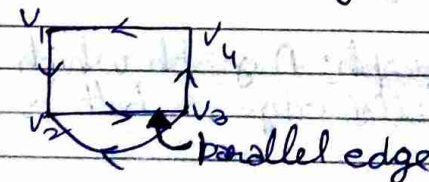
- (i) Weighted Graph: A weighted graph is a graph in which each branch is given a numerical weight.



- (ii) Self loop: If there is an edge whose starting & ending vertices is same.

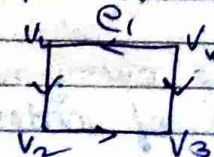


- (iii) Parallel Edges: If there are more than one edge between the same pair of vertices, they are known as parallel edges.



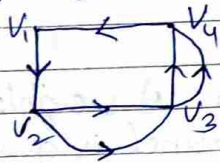
- (iv) Adjacent Vertices: A vertex V_1 is adjacent to vertex V_2 if there is an edge from V_1 to V_2 .

- (v) Incidence: In a graph the edge is incidence on vertex V_1 & V_2 .



e_1 is incidence on vertex V_1 & V_2

(vi) Degree of vertex: The degree of vertex is the number of edges incident to that vertex.



$$\begin{aligned}(V_2)_D &= 3 \\ (V_3)_D &= (V_4)_D = 3 \\ (V_1)_D &= 2\end{aligned}$$

There are two types of degrees:

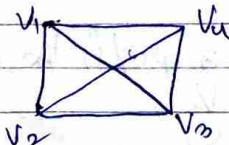
- (i) In-degree $\rightarrow$ Coming edges
- (ii) Out-degree $\rightarrow$ Going edges

Eg: In above graph in-degree of $V_2 = 1$
out-degree of $V_2 = 2$

(vii) Simple graph: A graph which doesn't have self loop or parallel edges then it is called simple graph.

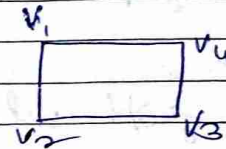
(viii) Multigraph: A graph which has self loop or parallel edge or both is called a multigraph.

(ix) Complete graph: If each vertex is adjacent to every other vertex in graph then it is called a complete graph.



(x) Regular graph: If every node is adjacent to the same no. of nodes then it is a regular graph.

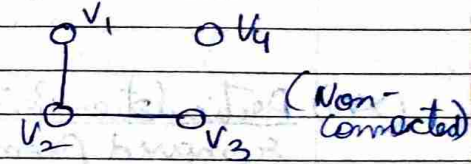
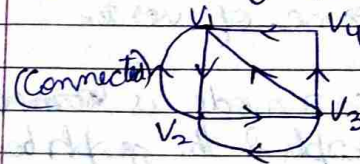
graph.



V_1 is adjacent to V_2 & V_4

(xi) Planar graph: A graph is planar if it can be drawn in a plane without intersecting two edges.

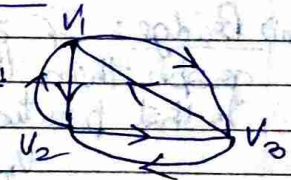
(xii) Connected graph:



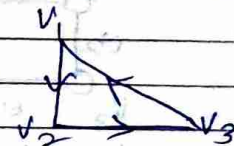
In a graph G , 2 vertices are said to be connected if there is a path from V_1 to V_2 or from V_2 to V_1 .

Types of Connected graph:

(a) Strongly Connected graph:



(b) Weakly Connected:



(xiii) Cycle: If there is a path containing one or more edges which start from a vertex & terminate onto the same vertex, then the path is called a cycle.

(xiv) Ayclic graph: A graph which doesn't have a cycle.

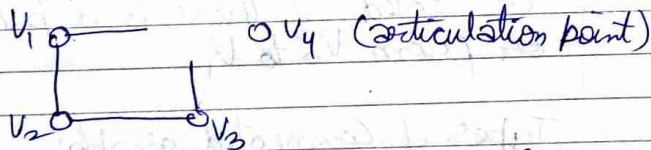
(xv) Cyclic graph: A graph which has a cycle.

(xvi) Maximum edges in the graph:

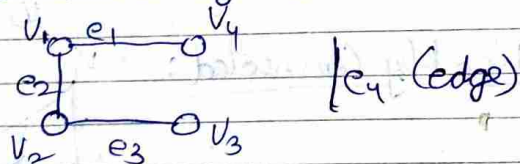
(a) Directed graph: $M.E = \frac{n(n-1)}{2}$

(b) Directed graph: $M.E = n(n-1)$
 $n \rightarrow$ no. of vertex

(xvii) Articulation Point: If a node is removed from the graph & the graph becomes disconnected, then the node is called the articulation point.



(xviii) Bridge: If an edge is removed from a graph then the graph becomes disconnected & that particular edge is called bridge.

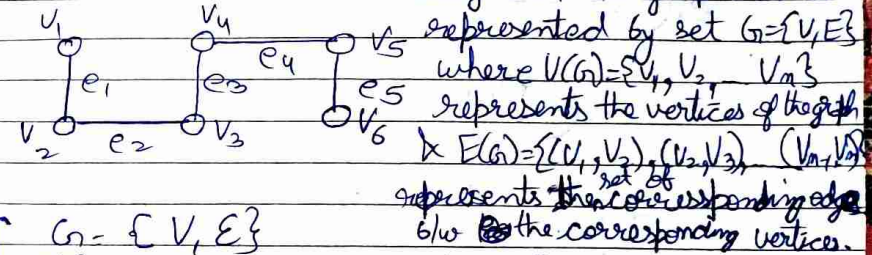


(xix) Biconnected Graph: A graph with no articulation point is called biconnected graph.

Representation of Graph:

1) Set Representation: Set representation of a graph $G = \{V, E\}$ consists of two sets V & E .

Set representation of a graph is a set of all of its vertices & the total no. of edges in the graph & it is represented by set $G = \{V, E\}$



where $V(G) = \{V_1, V_2, V_3, V_4, V_5, V_6\}$ represents the vertices of the graph & $E(G) = \{(V_1, V_2), (V_2, V_3), (V_3, V_4), (V_4, V_5), (V_5, V_6)\}$ represents the corresponding edges b/w the corresponding vertices.

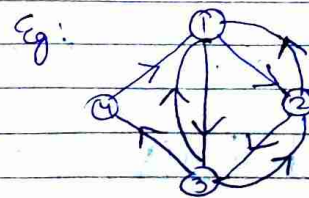
2) Sequential Representation:

(a) Adjacent Matrix $\rightarrow$ It is a 2D matrix of size $V \times V$ where V is the no. of vertices in a graph.

(b) Incident Matrix $\rightarrow$ It is a 2D matrix of size $V \times E$ where V is the no. of vertices & E is the total no. of edges.

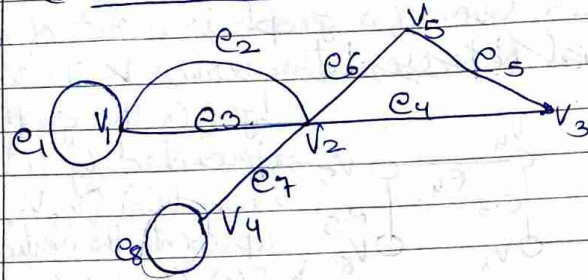
	V_1	V_2	V_3	V_4	V_5	...
V_1						
V_2						
V_3						
V_4						
V_5						

If there is an edge b/w any two vertices then represent it by 1 if not then 0.



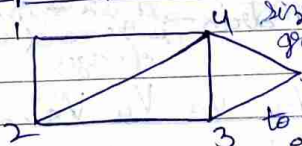
	1	2	3	4
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1
4	1	0	0	0

(b) Incidence Matrix:



	e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8
V_1	1	1	1	0	0	0	0	0
V_2	0	1	1	1	0	1	1	0
V_3	0	0	0	1	1	0	0	0
V_4	0	0	0	0	0	0	1	1
V_5	0	0	0	1	1	1	0	0

3.) Link Representation: In this representation, the list is the array of linked list, whose array size is same as no. of vertices in the graph. Every vertex has a linked list. Each node in this linked list represents the reference to the other vertices which share an edge with the current vertex.

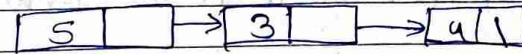
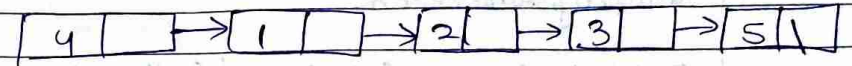
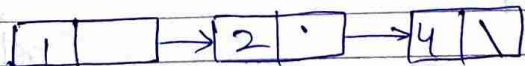


Weighted graph -

Weight	Vertex or info	Adj. vertex
--------	----------------	-------------

Non-Weighted graph -

Info	Adj. vertex
------	-------------



16.10.19 Traversal Algorithm in Graph:

- 1.) Breadth First Search (BFS)
- 2.) Depth First Search (DFS)

I Breadth First Search (BFS):

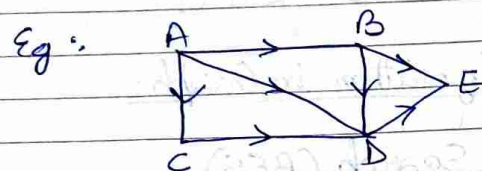
- (i) Insert the starting node in queue.
- (ii) Delete front element from queue & insert all its ^{adjacent} unvisited elements in to the queue. At the end, traverse back. Also make the value of visited array true for these nodes.
- (iii) Repeat Step (ii) until queue is empty.

II Depth First Search (DFS):

- (i) Push starting node into the stack.
- (ii) Pop an element from the ^{top of} stack, if it has not been traversed then traverse it otherwise ignore it. After traversing make the value of visited array true for these nodes.
- (iii) Now push all unvisited adjacent nodes of the popped element ^{into the stack}. Also push the element even if

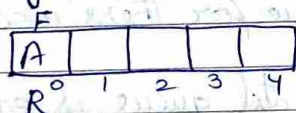
already in stack.

(iv) Repeat Step (ii) & (iii) until the stack is empty.



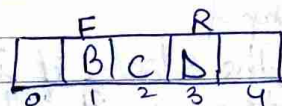
Vertex	Adjacently list
A	B C D
B	D E
C	D
D	E
E	-

(i) Initially push A into the queue.



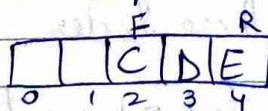
$F=R=0$

(ii) Remove front element & insert its all adjacent vertices into queue. If it is not in queue.



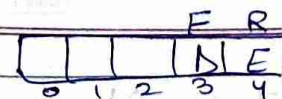
→ Traversed Node = A

(iii) Repeat Step (ii)



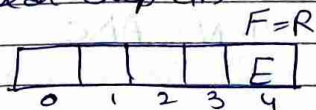
→ Traversed Node = AB

(iv) Repeat Step (ii)



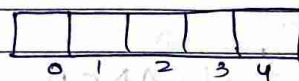
→ Traversed Node = ABC

(v) Repeat Step (ii)



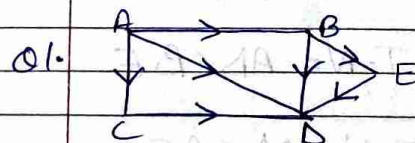
→ Traversed Node = ABCD

(vi) Repeat Step (ii)



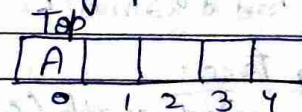
→ Traversed Node = ABCDE

$F=R=-1$

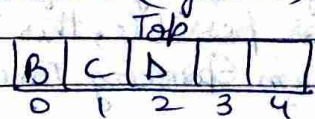


Vertex	Adjacently list
A	B C D
B	D E
C	D
D	-
E	D

(i) Initially push A into the stack

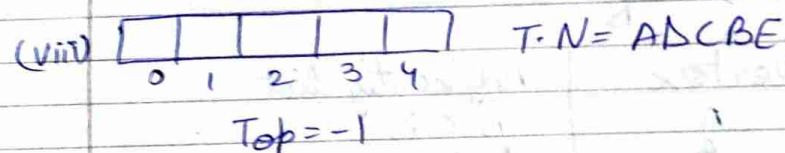
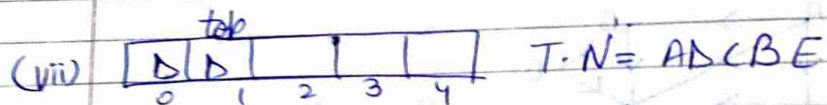
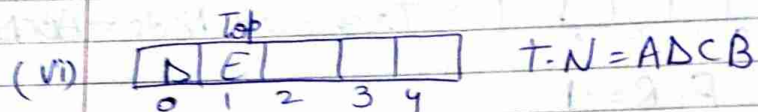
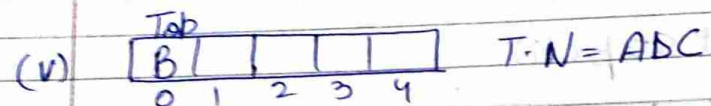
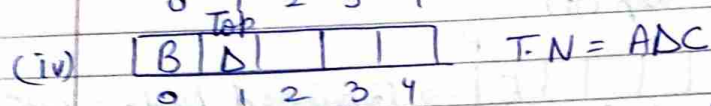
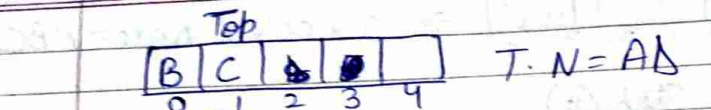


(ii) Pop the top element from stack & push its all vertices (adjacent) into the stack.



→ Traversed Node = A

(iii) Repeat Step (ii)



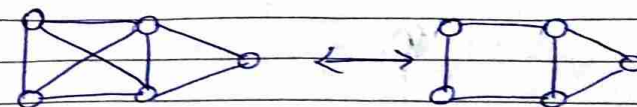
Spanning Tree:

A Spanning tree of a graph is just a sub graph that contain all vertices ^{with min. possible no. of edges}.
So, in this a number of edges will be one less than the number of nodes. Hence a S.T doesn't contain a cycle and it cannot be disconnected.

Minimum Spanning Tree:

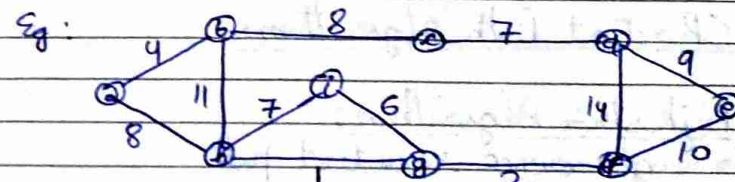
In a weighted connected graph (G), it is often desired to create a spanning tree (T) of graph (G), such that the sum of the weight of the tree edges is as small as possible, this type of tree is called the Minimum Spanning Tree.

For eg: Given a tree as shown below



Here no. of edges is one less than no. of vertices

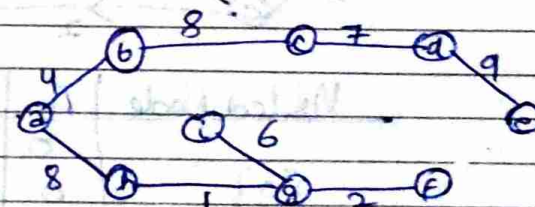
Kruskal's Algorithm:



Edge	Weight
(a,b)	4
(b,c)	11
(a,c)	8
(i,g)	6
(i,h)	7
(c,d)	7
(b,c)	8
(a,h)	8
(d,e)	9
(e,f)	10
(b,h)	11
(d,f)	14

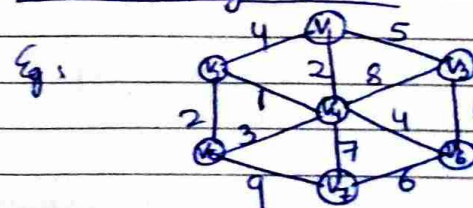
★ no cycle

★ (i,h) → cycle ignore



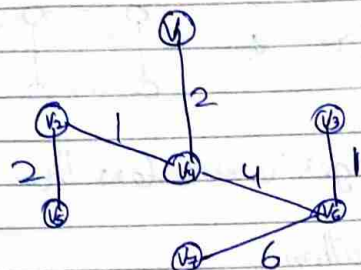
M-ST = 45 (by adding all the weight)

Prim's Algorithm:



★ No cycle

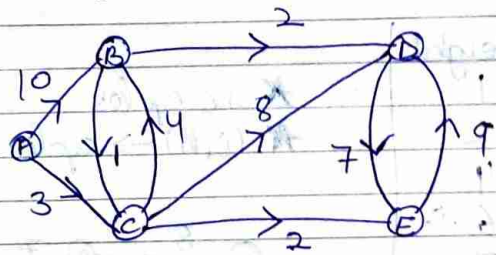
★ Min. weight



M-ST = 16

Shortest Path Algorithm:

- 1.) Dijkstra Algorithm:
→ single source shortest path



Visited Node	A	B	C	D	E
{A}	0	∞	∞	∞	∞
{A, C}	0	10	3	∞	∞
{A, C, E}	0	7	3	11	5
{A, C, E, B}	0	7	3	9	5
{A, C, E, B, D}	0	7	3	9	5

17.10.19

Unit-4 Searching and Sorting

- 1.) Linear Search or Sequential
- 2.) Binary Search Tree

Sorting:

- (i) Radix or Bucket Sorting:

342, 340, 128, 429, 523, 625, 90

(P.1) Elements	0	1	2	3	4	5	6	7	8	9	(P.2) Elements
342			(P.1) 342		(P.2) 340	(P.3) 523					
340	(P.1) 340						(P.3) 625			(P.2) 090	340
128		(P.3) 128			(P.2) 342				(P.1) 128		090
429			(P.2) 523		(P.3) 429					(P.1) 429	342
523			(P.2) 625	(P.1) 523							523
625			(P.2) 128	(P.3) 340		(P.1) 625					625
090	(P.1) 090	(P.3) 429									128

- Step-1: Pass 1 acc. to the last digit (from right hand side) put elements in respective column.
- Step-2: Place them starting from 0 column to 9
- Step-3: Repeat Step-1 for second digit (Pass 2)
- Step-4: Now acc. to 1st digit of the numbers place them in respective columns.

(ii) Selection Sort:

40, 50, 10, 90, 30

Pass-1 → Fix one value & compare. If it is greater than replace otherwise no change

40 50 10 90 30 (Replace 40 by 10)
10 50 40 90 30 (No change)

Pass-2: Same as Pass-1.

10 50 40 90 30 (Replace 50 by 40)
10 40 50 90 30 (Replace 40 by 30)

10 30 50 90 40 (No change)

Pass-3: 10 30 50 90 40 (Replace 50 by 40)
10 30 40 90 50 (No change)

Pass-4: 10 30 40 90 50 (Replace 90 by 50)

10 30 40 50 90 (Selection Sort)

★ If $n \rightarrow$ no. of elements in a list then in selection sort we get our solution in $n-1$ passes.

(iii) Insertion Sort:

$i=0$ 40, 50, 12, 30, 90, 18, 6, 60
For $i=2 \leftarrow 5$

Step-1: It will start from 2 position

Step-2: Compares 50 with 40 if lesser put element at every front.

For $i=3$ 12 40 50 30 90 18 6 60

$i=4$ 12 30 40 50 90 18 6 60

$i=5$ 12 18 30 40 50 90 6 60

$i=6$ 6 12 18 30 40 50 90 60

$i=7$ 6 12 18 30 40 50 60 90

(iv) Bubble Sort:

	Pass 1						Pass 2			
5	2	2	2	2	2	2	1	1	1	1
2	5	1	1	1	1	1	2	2	2	2
1	1	5	4	4	4	4	4	4	3	3
4	4	4	5	3	3	3	3	3	4	4
3	3	3	3	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7	7	7	7

(v) Quick Sort:

36 15 40 1 60 28 55 25 50 20

Pivot element $\rightarrow$ supposed (36) $\leftarrow$ Small
large $\rightarrow$ } Swap

$\rightarrow$ Right to Left ($\leftarrow$):

20 15 40 1 60 28 55 25 50 [36] $\leftarrow$

$\rightarrow$ Left to Right ($\rightarrow$):

$\rightarrow$ 20 15 [36] 1 60 28 55 25 50 40
20 15 25 1 60 28 55 [36] 50 40 $\leftarrow$
 $\rightarrow$ 20 15 25 1 [36] 28 55 60 50 40
20 15 25 1 28 [36] 55 60 50 40 $\leftarrow$

Fixed position

Again apply same procedure to the remaining elements.

21.10.19 # Program of Bubble Sort:

```
int a[] = {25, 20, 10, 5, 8, 15};
int i, j, temp;
for (i = 0; i < n; i++)
{
    for (j = 0; j < n-1; j++)
    {
        if (a[j] > a[j+1])
        {
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }
    }
}
```

Program of Selection Sort:

```
int a[] = {30, 25, 55, 20, 18, 15};
int i, j, temp;
for (i = 0; i < n-1; i++)
{
    for (j = i+1; j < n; j++)
    {
        if (a[i] > a[j])
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }
}
```

Program of Insertion Sort:

```
int a[] = {40, 35, 30, 32, 20, 15, 18};
int i, j, temp;
for (i = 1; i < n; i++)
{
    temp = a[i];
    for (j = i-1; j >= 0; j--)
    {
        if (temp < a[j])
        {
            a[j+1] = a[j];
        }
        else
        {
            break;
        }
    }
    a[j+1] = temp;
}
```